Kubernetes Zero-to-Advanced Roadmap

এই roadmap কীভাবে ব্যবহার করবেন

এটা ১৪টা Phase (P0–P13) আর ৬০টার মতো numbered module এর একটা roadmap; প্রতিদিন ~১ ঘণ্টা করে পড়লে core অংশ শেষ হতে **প্রায় ১০–১২ মাস** লাগবে। প্রতিটা module এর একটা unique ID আছে (যেমন **M2.1**), পরে "continue" বা "M3.2 শুরু করো" বললে ঠিক সেখান থেকে শুরু হবে।

Version note: Kubernetes বছরে ৩টা minor release দেয় এবং একসাথে শেষ ৩টা minor version support পায়। এই doc লেখার সময় (Oct 2026) সর্বশেষ minor version v1.37; kubernetes.io এর current docs এই version এর। কোনো feature version-sensitive হলে syllabus এ **[version-sensitive]** লেখা থাকবে; module পড়ার সময় আমরা kubernetes.io/releases এ গিয়ে মিলিয়ে নেব।

গুরুত্বপূর্ণ পরিবর্তন যেটা পুরনো tutorial এ পাবেন না: জনপ্রিয় Ingress NGINX controller **March 2026 এ retire** হয়েছে, আর Ingress API এখন frozen; নতুন development হচ্ছে **Gateway API** তে। তাই এই roadmap এ Ingress কে concept হিসেবে শেখানো হবে, কিন্তু hands-on traffic management হবে Gateway API দিয়ে। (Kubernetes blog)

Legend

চিহ্ন	মানে
L1 — Awareness	কী জিনিস, কেন আছে — বুঝলেই চলবে
L2 — Working Knowledge	নিজে configure ও ব্যবহার করতে পারবেন, normal workflow ব্যাখ্যা করতে পারবেন
L3 — Deep Understanding	internals, trade-off, failure mode, troubleshooting সহ
L4 — Specialist	production edge case ও design trade-off; platform engineer দের জন্য, আপনার জন্য সাধারণত postpone
[LAB-SAFE]	local cluster এ নির্ভয়ে চালাতে পারবেন
[PROD-CAUTION]	production এ চালালে outage/data loss হতে পারে — আগে context চেক, review, backup
[DOCKER-MIN]	Docker এর minimum concept, এখানেই শেখানো হবে

চিহ্ন	মানে
[DOCKER-TRACK]	আলাদা Docker track এ শিখবেন, নির্দিষ্ট exercise এর আগে
[OPTIONAL]	এখন skip করা যায়

Running example: পুরো roadmap জুড়ে একটা কাল্পনিক **"SubTrack" subscription service** বানাব — Node.js API + PostgreSQL + background worker + renewal CronJob + mock payment webhook। কোনো real payment credential বা real provider ব্যবহার হবে না।

A. Learning goal ও final capabilities

লক্ষ্য: roadmap শেষে আপনি একটা real Node.js + PostgreSQL application কে Kubernetes এ নিজে deploy, secure, scale, monitor ও troubleshoot করতে পারবেন, এবং production cluster এ কী হচ্ছে সেটা বুঝে team এর সাথে আলোচনা করতে পারবেন।

Roadmap শেষে আপনি যা পারবেন:

1. **Architecture ব্যাখ্যা:** `kubectl apply` থেকে শুরু করে container চালু হওয়া পর্যন্ত প্রতিটা component (API server, etcd, scheduler, controllers, kubelet) কী করে — whiteboard এ এঁকে বোঝাতে পারবেন।
2. **Deploy:** নিজের লেখা YAML দিয়ে Deployment, Service, ConfigMap, Secret, PVC, StatefulSet, Job, CronJob বানাতে পারবেন — copy না করে, প্রতিটা field কেন আছে জেনে।
3. **Networking:** Pod-to-Pod, Service, DNS ও Gateway API দিয়ে traffic কীভাবে যায় trace করতে পারবেন।
4. **Reliability:** probes, resource requests/limits, rolling update, rollback, HPA, PodDisruptionBudget দিয়ে zero-downtime deploy করতে পারবেন।
5. **Security:** RBAC, ServiceAccount, securityContext, Pod Security Standards, NetworkPolicy দিয়ে least-privilege setup করতে পারবেন।
6. **Delivery:** Helm chart লিখে, CI pipeline এ image build/push করে, GitOps (Argo CD) দিয়ে deploy automate করতে পারবেন।
7. **Troubleshooting:** CrashLoopBackOff, ImagePullBackOff, Pending, DNS failure, wrong selector, OOMKilled, failed rollout — প্রতিটার জন্য একটা systematic diagnosis workflow চালাতে পারবেন।
8. **Observability:** logs, metrics (Prometheus/Grafana), events দেখে সমস্যা বের করতে পারবেন।
9. **Production awareness:** managed Kubernetes (EKS/GKE/AKS), upgrade, backup/DR, cost — কোনটা আপনার দায়িত্ব আর কোনটা platform team এর, সেটা আলাদা করতে পারবেন।

Honest expectation: এই roadmap আপনাকে *foundational competence* দেবে — নিজে deploy ও debug করতে পারার মতো দক্ষতা। *Production experience* আসে real incident, real traffic আর real on-call থেকে; সেটা কোনো course শর্টকাটে দিতে পারে না। Roadmap শেষে আপনি একজন confident application-side Kubernetes user হবেন, "Kubernetes expert" হতে আরও ১–২ বছর real কাজ লাগবে।

B. আপনার বর্তমান জ্ঞান নিয়ে assumptions

বিষয়	ধরে নিচ্ছি	কী করব
JavaScript/TypeScript, Node.js	ভালো জানেন	Example app এগুলোতেই লেখা হবে
REST API, HTTP request/response	জানেন	HTTP status, header, port এর network দিকটা নতুন করে ব্যাখ্যা করব
Database (PostgreSQL)	query ও schema জানেন	Database কে "stateful workload" হিসেবে চালানো নতুন হবে
Git	basic জানেন	Branching, CI trigger, GitOps এ কাজে লাগবে
Linux command line	জানেন না ধরে নিচ্ছি	P0 তে minimum শেখানো হবে
Networking (IP, DNS, TCP, routing)	জানেন না ধরে নিচ্ছি	P0 তে basic, P3 তে Kubernetes context এ গভীরে
Docker / containers	জানেন না ধরে নিচ্ছি	Concept এখানে; hands-on Docker আলাদা track এ
Servers, VM, infrastructure	জানেন না ধরে নিচ্ছি	P0 তে concept
YAML	হয়তো দেখেছেন, গভীরে না	P0 তে syntax ও pitfalls
DevOps / CI/CD	concept হালকা	P9 তে পুরোটা

আপনার কাছে একটা laptop আছে (কমপক্ষে 8 GB RAM, 16 GB হলে ভালো) যেখানে Docker Desktop বা Podman install করা যাবে — local cluster এর জন্য এটা লাগবে। কোনো assumption ভুল হলে বলবেন, সেই অংশ ছোট বা বড় করে দেব।

C. Prerequisites ও আলাদা Docker learning track

সব prerequisite আগে master করতে হবে না — শুধু P0 এর minimum করে Kubernetes শুরু করবেন, বাকিটা যখন দরকার তখন (just-in-time) শিখব। নিচের table বলে কোন জিনিস কতটুকু আর কোথায় কাজে লাগবে।

Prerequisite map

Prerequisite	Minimum যা লাগবে	Kubernetes এ কোথায় কাজে লাগে	কখন শিখবেন
Linux command line	`ls`, `cd`, `cat`, `grep`, `curl`, pipe, file path	`kubectl exec` দিয়ে container এ ঢুকে debug	M0.1, শুরুতেই
Processes, PID, signals	process কী, `SIGTERM` vs `SIGKILL`, exit code	Pod shutdown, graceful termination, CrashLoopBackOff এর exit code	M0.2; গভীরে M5.1, M11.2
Ports	একটা process একটা port এ listen করে	`containerPort`, Service `port` vs `targetPort`	M0.2
Files & permissions	owner, `rwx`, root vs non-root user	`securityContext`, `runAsNonRoot`, volume permission error	M0.2; গভীরে M8.2
Environment variables	process কীভাবে env পড়ে	ConfigMap, Secret, `env` / `envFrom`	M0.2; গভীরে M3.4
IP address, subnet	private IP, CIDR (`10.0.0.0/16`) কী	Pod IP, Service ClusterIP, NetworkPolicy	M0.3; গভীরে M3.3
DNS	name থেকে IP resolve	CoreDNS, `my-svc.my-ns.svc.cluster.local`	M0.3; গভীরে M3.2
TCP, HTTP	connection, request, status code	probes, Gateway API routing	M0.3
Routing, NAT, load balancer	traffic এক জায়গা থেকে আরেক জায়গায় কীভাবে যায়	kube-proxy, Service, LoadBalancer	M0.3; গভীরে M3.3
Server, VM	physical vs virtual machine	Node কী জিনিস	M0.4

Prerequisite	Minimum যা লাগবে	Kubernetes এ কোথায় কাজে লাগে	কখন শিখবেন
Container concept	isolated process + image	Pod এর ভেতরে কী চলে	M0.4
YAML	indentation, map, list, multi-line string	প্রতিটা Kubernetes manifest	M0.5
Shell scripting	variable, loop, simple script	CI pipeline, debugging loop	M0.5 (basic)
Git & build/deploy	commit, branch, build artifact, release	CI/CD, GitOps	M0.6; গভীরে P9

Docker track — আলাদা রাখছি

এই roadmap Docker course না। Docker কে তিন ভাগে ভাগ করা হলো:

Docker topic	Tag	কখন লাগবে
Container কী, image কী, registry কী, image tag	[DOCKER-MIN]	M0.4 তে এখানেই শেখানো হবে
Container আর VM এর পার্থক্য	[DOCKER-MIN]	M0.4
Image immutable, container ephemeral (মুছে গেলে data যায়)	[DOCKER-MIN]	M0.4; storage বুঝতে জরুরি
Dockerfile লেখা (Node.js app এর জন্য)	[DOCKER-TRACK]	**Project 2 (M2.2) এর আগে** শিখতে হবে
`docker build`, `docker run`, `docker push`, port mapping	[DOCKER-TRACK]	Project 2 এর আগে
Multi-stage build, ছোট image, `.dockerignore`	[DOCKER-TRACK]	P9 (CI/CD) এর আগে
Non-root user দিয়ে image, minimal base image	[DOCKER-TRACK]	M8.2 (securityContext) এর আগে
Docker Compose	[OPTIONAL]	Local dev এর জন্য ভালো, কিন্তু Kubernetes বুঝতে লাগবে না

Docker topic	Tag	কখন লাগবে
Docker networking, volumes এর internals	[OPTIONAL]	Kubernetes এর নিজস্ব networking/storage model আছে; skip করা যায়
Docker Swarm	[OPTIONAL]	শিখবেন না — Kubernetes এর বিকল্প, এখন খুব কম ব্যবহৃত

Docker track এর জন্য সময়: ~১০–১৫ ঘণ্টা (২–৩ সপ্তাহ, দিনে ১ ঘণ্টা)। এটা P0 এর সাথে parallel বা P1 এর সময় করে ফেলুন, যাতে M2.2 তে পৌঁছানোর আগে নিজের Node.js app এর image বানাতে পারেন। একটা জিনিস মনে রাখবেন: Kubernetes এখন Docker Engine কে সরাসরি runtime হিসেবে ব্যবহার করে না (containerd/CRI-O ব্যবহার করে), কিন্তু Docker দিয়ে বানানো image (OCI image) Kubernetes এ ঠিকই চলে।

C+. Setup: installation, local cluster ও environment

এই অংশ শেষে আপনার laptop এ একটা ৩-node (১ control plane + ২ worker) kind cluster চলবে, সব tool install থাকবে, আর lab এর জন্য আলাদা namespace ও folder তৈরি থাকবে। সময় লাগবে ~২–৩ ঘণ্টা; এটা M1.4 এর hands-on অংশ, তবে Docker track শুরুর আগেই tool install সেরে রাখতে পারেন। নিচের সব command **[LAB-SAFE]** — শুধু নিজের laptop এ।

Versions (Oct 2026 এ যাচাই করা): kind এর stable release **v0.33.0**; kubernetes.io এর current docs **v1.37**; Helm এর install script এখন `get-helm-4`। install করার দিন আবার মিলিয়ে নেবেন — এগুলো দ্রুত বদলায়।

ধাপ ১ — Hardware চেক

Resource	Minimum	Recommended	কেন
RAM	8 GB	16 GB	প্রতিটা kind node একটা container; ৩ node + monitoring stack এ 6–8 GB লাগে
CPU	4 core	6–8 core	control plane + app + load test একসাথে
Free disk	30 GB	50 GB+	node image, app image, Prometheus data

Resource	Minimum	Recommended	কেন
Docker এ বরাদ্দ (Docker Desktop)	4 CPU / 6 GB	6 CPU / 8 GB+	Docker Desktop এর Settings → Resources থেকে বাড়াতে হয়

RAM 8 GB হলে ৩-node এর বদলে ২-node (১ control plane + ১ worker) দিয়ে শুরু করবেন; node failure lab এর সময় তৃতীয় node যোগ করব।

ধাপ ২ — Operating system অনুযায়ী ভিত্তি

আপনার OS	কী করবেন	কোথায় command চালাবেন
Windows 10/11	PowerShell (Admin) এ `wsl --install -d Ubuntu`; restart; তারপর Docker Desktop install করে Settings → Resources → WSL Integration এ Ubuntu চালু	সব command **Ubuntu (WSL2) terminal** এ — PowerShell এ না। এতে roadmap এর সব Linux command হুবহু কাজ করবে
macOS	Homebrew install; Docker Desktop (বা OrbStack / Colima — হালকা বিকল্প)	Terminal (zsh)
Linux (Ubuntu)	Docker এর official apt repo থেকে Docker Engine; তারপর `sudo usermod -aG docker $USER` আর logout/login	যেকোনো terminal

সতর্কতা: Linux এ `docker` group এ থাকা মানে কার্যত root access — শুধু নিজের ব্যক্তিগত/lab machine এ করবেন। Docker Desktop বড় কোম্পানিতে paid license চাইতে পারে; অফিসের machine হলে IT/policy জেনে নিন, না হলে Podman বা Colima ব্যবহার করুন (kind দুটোই চিনতে পারে)।

Docker ঠিক আছে কিনা যাচাই:

```
docker version          # Client আর Server দুটোই দেখাতে হবে
docker run --rm hello-world
```

ধাপ ৩ — Tools তালিকা

Tool	কাজ	কখন লাগবে	প্রয়োজনীয়তা
Docker Engine/Desktop (বা Podman)	container চালানো; kind এর node এগুলোই	এখনই	Required

Tool	কাজ	কখন লাগবে	প্রয়োজনীয়তা
kubectl	cluster এর সাথে কথা বলার CLI	এখনই	Required
kind	local cluster বানানো	এখনই	Required
Helm	package manager	M9.2 (add-on install এর জন্য আগেও লাগতে পারে)	Required (পরে)
kube-ps1	shell prompt এ current context ও namespace দেখায়	এখনই	**Strongly recommended** — ভুল cluster এ command চালানো আটকায়
kubectx + kubens	context/namespace দ্রুত বদলানো	M1.5	Optional
k9s	terminal UI দিয়ে cluster দেখা	M1.5 এর পরে	Optional — আগে kubectl শিখুন, তারপর k9s
jq, yq	JSON/YAML থেকে value বের করা	M0.5	Recommended
stern	একাধিক Pod এর log একসাথে	M10.3	Optional
VS Code + "YAML" (Red Hat) ও "Kubernetes" extension	YAML validation, schema autocomplete	এখনই	Recommended
Node.js LTS, Git	app ও version control	আছে	Required

ধাপ ৪ — Install commands

macOS (Homebrew):

```
brew install kubectl kind helm kube-ps1 kubectx k9s jq yq
```

Linux / WSL2 Ubuntu (x86_64):

```
# kubectl — official binary + checksum যাচাই
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl.sha256"
echo "$(cat kubectl.sha256)  kubectl" | sha256sum --check   # "kubectl: OK"
আসতে হবে
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
rm kubectl kubectl.sha256

# kind v0.33.0
curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.33.0/kind-linux-amd64
chmod +x ./kind
sudo mv ./kind /usr/local/bin/kind

# Helm 4 — script টা আগে পড়ে নিন, তারপর চালান
curl -fsSL -o get_helm.sh
https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-4
less get_helm.sh
chmod 700 get_helm.sh && ./get_helm.sh

# jq, yq ও kubectl autocompletion
sudo apt-get update && sudo apt-get install -y jq
sudo snap install yq      # snap না থাকলে GitHub release থেকে binary
echo 'source <(kubectl completion bash)' >> ~/.bashrc
```

ARM64 machine (যেমন ARM Linux) হলে URL এ `amd64` এর জায়গায় `arm64` দেবেন। Windows এ native PowerShell এ চালাতে চাইলে `winget install Kubernetes.kind` ও `winget install Helm.Helm` আছে, কিন্তু roadmap WSL2 ধরে লেখা — সেটাই ব্যবহার করুন।

Install যাচাই:

```
kubectl version --client
kind version            # v0.33.0
helm version            # v4.x
docker info | grep -E "CPUs|Total Memory"   # Docker এ কত resource দেওয়া আছে
```

kubectl version নিয়ম: kubectl cluster এর version থেকে সর্বোচ্চ ১ minor version এদিক-ওদিক হতে পারে (যেমন v1.37 client → v1.36, v1.37, v1.38 cluster)। অফিসের cluster পুরনো হলে সেটার জন্য আলাদা kubectl version লাগতে পারে।

ধাপ ৫ — Lab folder আর kind cluster config

সব কিছু একটা Git repo তে রাখব, যাতে cluster মুছে গেলেও এক command এ আবার বানানো যায়:

```
mkdir -p ~/k8s-lab/{cluster,manifests,apps,notes,scripts}
cd ~/k8s-lab && git init
```

`~/k8s-lab/cluster/kind-config.yaml`:

```
# 1 control plane + 2 worker — scheduling ও node failure practice এর জন্য
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
name: subtrack-lab
nodes:
- role: control-plane
  extraPortMappings:        # NodePort Service laptop থেকে access করার জন্য
(M3.1)
  - containerPort: 30080
    hostPort: 30080
  - containerPort: 30443
    hostPort: 30443
- role: worker
- role: worker
```

কেন এভাবে: `name` দিলে context হবে `kind-subtrack-lab` (চেনা সহজ); ২টা worker থাকায় anti-affinity আর node failure দেখা যাবে; `extraPortMappings` ছাড়া NodePort laptop থেকে পৌঁছায় না, কারণ kind node আসলে Docker container। Kubernetes version kind এর default node image থেকে আসে; নির্দিষ্ট version চাইলে kind release notes থেকে `kindest/node:vX.Y.Z@sha256:...` image প্রতিটা node এ `image:` field এ দেবেন (sha সহ — tag একা দেবেন না)।

ধাপ ৬ — Cluster তৈরি ও যাচাই

```
kind create cluster --config ~/k8s-lab/cluster/kind-config.yaml --wait 5m

kubectl config current-context        # kind-subtrack-lab
kubectl cluster-info
kubectl get nodes -o wide             # ৩টা node, সব Ready
kubectl get pods -n kube-system       # coredns, etcd, kube-apiserver,
kindnet, kube-proxy ...
kubectl get --raw='/readyz?verbose'   # সব check "ok"
docker ps                             # ৩টা node আসলে ৩টা container — নিজের
চোখে দেখুন
```

Smoke test — একটা Pod চালিয়ে, browser এ দেখে, মুছে ফেলা:

```
kubectl run hello --image=nginx:1.27 --restart=Never
kubectl wait --for=condition=Ready pod/hello --timeout=120s
kubectl port-forward pod/hello 8080:80     # browser: http://localhost:8080 ,
Ctrl+C দিয়ে থামান
kubectl delete pod hello
```

এটা কাজ করলে cluster, image pull, scheduling, kubelet, networking — সব মূল অংশ ঠিক আছে।

ধাপ ৭ — নিজের image cluster এ পাঠানো

kind cluster laptop এর Docker image সরাসরি দেখতে পায় না; image আলাদা করে load করতে হয় [DOCKER-TRACK এর পরে লাগবে]:

```
docker build -t subtrack-api:0.1.0 ~/k8s-lab/apps/subtrack-api
kind load docker-image subtrack-api:0.1.0 --name subtrack-lab
```

`latest` tag কখনো ব্যবহার করবেন না: `latest` হলে Kubernetes এর default pull policy `Always`, তাই node এ load করা image না নিয়ে registry থেকে টানতে গিয়ে ImagePullBackOff হবে। প্রতিবার নতুন build এ নতুন tag (`0.1.1`, বা git SHA)। পরে (M9.4) kind এর local registry setup করব।

ধাপ ৮ — Add-ons: শুধু যখন module এ লাগবে তখন

শুরুতেই সব install করবেন না — কোন জিনিস কী সমস্যা সমাধান করে সেটা module এ শেখার পর install করলে বুঝে করবেন।

Add-on	কেন	কোন module এ install	মন্তব্য
metrics-server	`kubectl top`, HPA	M5.2 / M5.5	kind এ `--kubelet-insecure-tls` flag লাগে — শুধু lab এ, production এ কখনো না [PROD-CAUTION]
cloud-provider-kind	`type: LoadBalancer` Service কে IP দেওয়া	M3.1 / M6.2	kind LoadBalancer guide অনুসরণ
Gateway API CRDs + একটা controller (যেমন Envoy Gateway)	external HTTP traffic	M6.2	তখনকার current version দেখে
cert-manager	TLS certificate	M6.3	Helm দিয়ে
Calico বা Cilium (NetworkPolicy সমর্থিত CNI)	NetworkPolicy lab	M8.4	তখন CNI support যাচাই করে নতুন cluster বানাতে হতে পারে
kube-prometheus-stack, Loki	monitoring, logging	M10.3 / M10.4	RAM বেশি খায় — 16 GB machine ভালো
Argo CD	GitOps	M9.5	Helm দিয়ে

metrics-server এর command টা আগেই রেখে দিচ্ছি (M5.2 তে চালাবেন):

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
kubectl patch -n kube-system deployment metrics-server --type=json \
  -p '[{"op":"add","path":"/spec/template/spec/containers/0/args/-","value":"--kubelet-insecure-tls"}]'
kubectl top nodes     # ১–২ মিনিট পরে data আসবে
```

ধাপ ৯ — Lab environment: namespace ও context

Namespace গুলো YAML file এ রাখি (declarative), command দিয়ে না — যাতে reset এর পর হুবহু ফেরত আসে। `~/k8s-lab/cluster/namespaces.yaml`:

```
apiVersion: v1
kind: Namespace
metadata:
  name: lab              # প্রতিদিনের পরীক্ষা, ভাঙা-গড়া
  labels:
    purpose: scratch
---
apiVersion: v1
kind: Namespace
metadata:
  name: subtrack-dev     # SubTrack project এর "dev" environment
  labels:
    app.kubernetes.io/part-of: subtrack
    environment: dev
---
apiVersion: v1
kind: Namespace
metadata:
  name: subtrack-prod    # production-এর মতো কড়া নিয়ম (P8 থেকে security এখানে
enforce করব)
  labels:
    app.kubernetes.io/part-of: subtrack
    environment: prod-sim
```

```
kubectl apply -f ~/k8s-lab/cluster/namespaces.yaml
kubectl config set-context --current --namespace=lab   # default namespace =
lab
kubectl get ns --show-labels
```

Context safety (সবচেয়ে গুরুত্বপূর্ণ অভ্যাস): আপনার laptop এ যদি অফিসের কোনো cluster এর kubeconfig থাকে, lab এর command ভুল করে সেখানে চলে যেতে পারে [PROD-CAUTION]। তাই:

```
kubectl config get-contexts     # কোন কোন cluster আপনার kubeconfig এ আছে
দেখুন
```

- অফিসের context থাকলে lab এর জন্য আলাদা kubeconfig file ব্যবহার করুন: cluster বানানোর সময় `--kubeconfig ~/.kube/kind-lab.yaml` দিন, আর lab terminal এ `export KUBECONFIG=~/.kube/kind-lab.yaml`।
- Prompt এ context দেখান (kube-ps1): bash এ `~/.bashrc` তে `source "<kube-ps1 path>/kube-ps1.sh"` আর `PS1='$(kube_ps1) '$PS1` (path install পদ্ধতি অনুযায়ী আলাদা; brew হলে `brew info kube-ps1` দেখায়)।

- ছোট alias: `alias k=kubectl` আর `complete -o default -F __start_kubectl k` (autocompletion alias এও কাজ করবে)।
- নিয়ম: যেকোনো `delete`, `drain`, `scale` এর আগে prompt এ context দেখে নেবেন।

ধাপ ১০ — Project folder structure

```
k8s-lab/
├── cluster/
│   ├── kind-config.yaml
│   └── namespaces.yaml
├── manifests/          # module অনুযায়ী YAML: m2.1-pod.yaml, m3.1-
service.yaml ...
├── apps/
│   └── subtrack-api/   # Node.js code + Dockerfile (Docker track এ বানাবেন)
├── notes/              # প্রতিটা module এর নোট ও review answers
├── scripts/
│   └── reset-lab.sh
└── .gitignore          # secrets/, *.env, kubeconfig — কখনো commit না
```

`.gitignore` এ অন্তত:

```
secrets/
*.env
*.kubeconfig
kind-lab.yaml
```

`~/k8s-lab/scripts/reset-lab.sh` — cluster ভেঙে গেলে বা পরিষ্কার শুরু চাইলে:

```
#!/usr/bin/env bash
set -euo pipefail
cd "$(dirname "$0")/.."
kind delete cluster --name subtrack-lab
kind create cluster --config cluster/kind-config.yaml --wait 5m
kubectl apply -f cluster/namespaces.yaml
kubectl config set-context --current --namespace=lab
kubectl wait --for=condition=Ready nodes --all --timeout=180s   # --wait শুধু
control-plane এর জন্য অপেক্ষা করে; worker দের জন্য এটা
kubectl get nodes
```

`chmod +x scripts/reset-lab.sh` করে রাখুন। এই script শুধু kind cluster মোছে — তাই lab-safe; কিন্তু ভেতরের সব data (PVC সহ) চলে যাবে, সেটা জেনে চালাবেন।

ধাপ ১১ — দৈনন্দিন ব্যবহার ও cleanup

কাজ	Command
Cluster গুলোর তালিকা	`kind get clusters`
পড়া শেষে RAM খালি করা	Docker Desktop বন্ধ করা যায়; আবার চালু করলে kind cluster সাধারণত ফিরে আসে, না এলে `scripts/reset-lab.sh`
Cluster মুছে ফেলা	`kind delete cluster --name subtrack-lab`
Cluster এর সব log export (debug এর জন্য)	`kind export logs ./kind-logs --name subtrack-lab`
Docker এর অব্যবহৃত image/cache মোছা	`docker system prune` — অব্যবহৃত সব image যাবে, আবার pull/build লাগবে

ধাপ ১২ — Setup troubleshooting

Symptom	সম্ভাব্য কারণ	সমাধান
`Cannot connect to the Docker daemon`	Docker চালু নেই; WSL এ Docker Desktop এর WSL integration বন্ধ	Docker চালু করুন; Settings → Resources → WSL Integration এ Ubuntu চালু; Linux এ docker group এর পর logout/login
`kind create cluster` আটকে থাকে বা fail করে	Docker এ RAM/CPU কম	Docker Desktop এ resource বাড়ান; ২-node config দিয়ে চেষ্টা
Linux এ Pod `too many open files` বা kube-proxy crash	inotify limit কম (kind known issues)	known issues page এর `fs.inotify` sysctl মান বসান
`port is already allocated`	hostPort 30080/30443 অন্য কিছু ব্যবহার করছে	config এ অন্য hostPort দিন বা সেই process বন্ধ করুন
`connection to the server ... was refused`	cluster মুছে গেছে বা ভুল context	`kubectl config current-context`, `kind get clusters`

Symptom	সম্ভাব্য কারণ	সমাধান
নিজের image এ ImagePullBackOff	`kind load` করেননি, বা `latest` tag	`kind load docker-image ... --name subtrack-lab`; versioned tag
`kubectl` আর cluster এর version এ warning	version skew > ১ minor	cluster এর কাছাকাছি kubectl version install

Setup checklist

- [ ] `docker run --rm hello-world` কাজ করে
- [ ] `kubectl version --client`, `kind version`, `helm version` সব দেখায়
- [ ] `kind-subtrack-lab` cluster এ ৩টা node Ready
- [ ] Smoke test Pod browser এ দেখা গেছে ও মোছা হয়েছে
- [ ] `lab`, `subtrack-dev`, `subtrack-prod` namespace আছে; default namespace `lab`
- [ ] Prompt এ current context দেখা যায়; অফিসের context lab থেকে আলাদা
- [ ] `~/k8s-lab` Git repo, `.gitignore` সহ; `reset-lab.sh` একবার চালিয়ে দেখা হয়েছে

Setup এর sources: kind Quick Start · Install kubectl on Linux · Installing Helm

D. সম্পূর্ণ roadmap index

Roadmap এ ১৪টা phase আর ৭৩টা module আছে, চারটা skill tier এ ভাগ করা: **Core** (P0–P5) সবার জন্য বাধ্যতামূলক, **Intermediate** (P6–P10) application deploy ও maintain করতে লাগে, **Advanced** (P11) production এ দরকার, **Specialist** (P12) বেশিরভাগ platform engineer দের জন্য — আপনি বেছে বেছে করবেন।

ID	Module	Depth	Tier
P0	**Foundations (just-in-time prerequisites)**		
M0.1	Linux ও command line essentials	L2	Core
M0.2	Processes, signals, ports, files, permissions, env vars	L2	Core
M0.3	Networking basics: IP, CIDR, DNS, TCP/HTTP, routing, NAT, load balancer	L2	Core

ID	Module	Depth	Tier
M0.4	Servers, VMs, containers, images, registries [DOCKER-MIN]	L2	Core
M0.5	YAML ও basic shell scripting	L2	Core
M0.6	Git, build artifact ও deployment concepts (12-factor app)	L2	Core
P1	**Kubernetes Fundamentals**		
M1.1	Kubernetes কেন: orchestration problem, declarative model, desired state	L3	Core
M1.2	Architecture: control plane ও worker node components	L3	Core
M1.3	Reconciliation loop ও request flow (kubectl থেকে container পর্যন্ত)	L3	Core
M1.4	Local cluster setup: kind, minikube, k3d, Docker Desktop তুলনা	L2	Core
C+	Setup: tool installation, kind cluster ও lab environment (M1.4 এর hands-on, ~২–৩ ঘণ্টা; tool install আগেই সেরে রাখা যায়)	L2	Core
M1.5	kubectl essentials, kubeconfig, contexts	L3	Core
M1.6	API objects, manifest anatomy, imperative vs declarative	L3	Core
P2	**Core Workloads**		
M2.1	Pods: lifecycle, restartPolicy, multi-container, init ও sidecar containers	L3	Core
M2.2	ReplicaSets ও Deployments	L3	Core
M2.3	Labels, selectors, annotations	L3	Core
M2.4	Namespaces ও resource organization	L2	Core
M2.5	Rolling updates, rollout history, rollback	L3	Core
P3	**Networking ও Configuration**		

ID	Module	Depth	Tier
M3.1	Services: ClusterIP, NodePort, LoadBalancer, headless; EndpointSlices	L3	Core
M3.2	DNS ও service discovery (CoreDNS)	L3	Core
M3.3	Cluster networking model: CNI, kube-proxy ও alternatives	L2	Core
M3.4	ConfigMaps ও environment variables	L3	Core
M3.5	Secrets ও secret handling basics	L3	Core
P4	**Storage ও Stateful Workloads**		
M4.1	Volumes: emptyDir, configMap/secret volume, projected	L2	Core
M4.2	PersistentVolume, PVC, StorageClass, CSI, dynamic provisioning	L3	Core
M4.3	StatefulSets ও headless Service	L2	Core
M4.4	PostgreSQL on Kubernetes vs managed database	L2	Core
P5	**Health, Resources ও Scheduling**		
M5.1	Probes: liveness, readiness, startup	L3	Core
M5.2	Requests, limits, QoS class, OOMKilled, CPU throttling	L3	Core
M5.3	LimitRange ও ResourceQuota	L2	Core
M5.4	Scheduling: nodeSelector, affinity, taints/tolerations, topology spread	L2	Core
M5.5	Scaling ও self-healing: manual scale, HPA, metrics-server	L2	Core
M5.6	Availability: PodDisruptionBudget, graceful shutdown, preStop	L3	Core
P6	**External Traffic Management**		
M6.1	Ingress: concept, controller ও বর্তমান status	L2	Intermediate
M6.2	Gateway API: GatewayClass, Gateway, HTTPRoute	L2	Intermediate

ID	Module	Depth	Tier
M6.3	TLS ও cert-manager	L2	Intermediate
P7	**Specialized Workloads**		
M7.1	Jobs (DB migration pattern সহ)	L2	Intermediate
M7.2	CronJobs	L2	Intermediate
M7.3	DaemonSets	L1	Intermediate
P8	**Security**		
M8.1	Authentication, authorization ও RBAC	L2	Intermediate
M8.2	ServiceAccounts ও workload identity	L2	Intermediate
M8.3	securityContext ও Pod Security Standards	L2	Intermediate
M8.4	NetworkPolicies	L2	Intermediate
M8.5	Production secrets: encryption at rest, External Secrets, Sealed Secrets	L2	Intermediate
M8.6	Supply-chain security: image scanning, signing	L1	Intermediate
P9	**Packaging ও Delivery**		
M9.1	Kustomize	L2	Intermediate
M9.2	Helm: chart ব্যবহার করা	L2	Intermediate
M9.3	Helm: নিজের chart লেখা	L2	Intermediate
M9.4	CI/CD pipeline (GitHub Actions: test, build, push, deploy)	L2	Intermediate
M9.5	GitOps: Argo CD (বা Flux)	L2	Intermediate
M9.6	Progressive delivery: blue-green, canary	L1	Intermediate
P10	**Troubleshooting ও Observability**		
M10.1	Troubleshooting methodology ও debugging toolkit	L3	Intermediate
M10.2	Failure scenario lab (৯টা real failure)	L3	Intermediate
M10.3	Logging: structured logs, centralized logging	L2	Intermediate

ID	Module	Depth	Tier
M10.4	Metrics, monitoring ও alerting (Prometheus, Grafana)	L2	Intermediate
M10.5	Tracing ও OpenTelemetry	L1	Intermediate
P11	**Production ও Cloud Operations**		
M11.1	Managed Kubernetes (EKS/GKE/AKS/DOKS) ও shared responsibility	L2	Advanced
M11.2	High availability ও reliability design	L2	Advanced
M11.3	Node pools ও cluster autoscaling (Cluster Autoscaler, Karpenter)	L1	Advanced
M11.4	Upgrades, version skew, deprecated API migration	L2	Advanced
M11.5	Backup ও disaster recovery	L2	Advanced
M11.6	Cost optimization	L1	Advanced
M11.7	Multi-environment setup (dev/staging/prod)	L2	Advanced
P12	**Advanced ও Specialist (optional, বেছে নেবেন)**		
M12.1	Advanced autoscaling: VPA, KEDA, in-place Pod resize	L1	Specialist
M12.2	Policy as code: ValidatingAdmissionPolicy, Kyverno	L1	Specialist
M12.3	Infrastructure as Code: Terraform/OpenTofu	L1	Specialist
M12.4	CRDs ও Operators	L1	Specialist
M12.5	Service mesh (Istio, Linkerd)	L1	Specialist
M12.6	kubeadm দিয়ে cluster from scratch, certificates, etcd operations	L4	Specialist
M12.7	Networking internals: CNI plugins, eBPF/Cilium	L4	Specialist
M12.8	Multi-cluster, multi-tenancy, GPU/DRA scheduling	L1	Specialist
P13	**Capstone**		
M13.1	Capstone architecture design	L3	Capstone
M13.2	Capstone build ও deploy	L3	Capstone

ID	Module	Depth	Tier
M13.3	Failure drills ও team presentation	L3	Capstone

E. Module-by-module syllabus

প্রতিটা module এর syllabus এখানে সংক্ষেপে: Purpose, Core concepts, Internals, Commands, Hands-on, Common mistakes, Troubleshooting, Production relevance, Done when।
পড়ানোর সময় প্রতিটা module এই ১০ ধাপে চলবে:

1. Learning objectives
2. Concept ও explanation (assigned depth অনুযায়ী)
3. প্রতিটা topic এর depth
4. Commands ও YAML examples
5. Guided hands-on lab
6. Independent practice task (আপনি নিজে করবেন, আমি review করব)
7. Debugging exercise (আমি কিছু ভাঙব, আপনি ঠিক করবেন)
8. Review questions
9. Team কে ৫–১০ মিনিটে শেখানোর presentation outline
10. Checkpoint — "Done when" পূরণ হলেই পরের module

Phase 0 — Foundations

M0.1 Linux ও command line essentials · L2

- **Purpose:** প্রতিটা container ভেতরে একটা ছোট Linux environment; debug করতে হলে shell এ চলাফেরা জানতে হবে।
- **Core concepts:** filesystem tree (`/` , `/etc` , `/var/log` , `/tmp`), absolute vs relative path, stdin/stdout/stderr, pipe (`|`), redirect (`>`), exit code।
- **Commands:** `ls` , `cd` , `pwd` , `cat` , `less` , `grep` , `find` , `ps` , `env` , `curl` , `wget` , `nslookup` / `dig` , `which` , `echo $?` ।
- **Hands-on:** WSL/macOS terminal এ একটা log file থেকে `grep` দিয়ে error line বের করা; `curl` দিয়ে নিজের Node.js API hit করা।
- **Common mistakes:** container এ `bash` না থাকায় অবাক হওয়া (অনেক image এ শুধু `sh` থাকে, কিছুতে কোনো shell নেই)।

- **Production relevance:** `kubectl exec -it pod -- sh` দিয়ে ঢুকে এই commands ই চালাবেন।
- **Done when:** কোনো GUI ছাড়া একটা file খুঁজে, তার ভেতর থেকে pattern বের করে, ফলাফল আরেক file এ লিখতে পারেন।

M0.2 Processes, signals, ports, files, permissions, env vars · L2

- **Purpose:** container আসলে একটা isolated process — process কীভাবে চলে, থামে, port খোলে, env পড়ে, সেটাই Pod behavior বুঝার ভিত্তি।
- **Core concepts:** PID ও PID 1; `SIGTERM` (ভদ্রভাবে থামো) vs `SIGKILL` (জোর করে মারো); exit code (0 = success, 1 = app error, 137 = SIGKILL/OOM, 143 = SIGTERM); port binding (`0.0.0.0` vs `127.0.0.1`); user/group, `rwx`, root vs non-root; `process.env`।
- **Hands-on:** Node.js app এ `process.on('SIGTERM')` handler লিখে `kill -TERM <pid>` দিয়ে test; app কে `127.0.0.1` এ bind করে বাইরে থেকে connect fail হতে দেখা।
- **Common mistakes:** container এ app `localhost` এ listen করা (বাইরে থেকে পৌঁছায় না); SIGTERM handle না করা।
- **Production relevance:** CrashLoopBackOff এর exit code, rolling update এ dropped request, non-root security — সবই এখান থেকে আসে।
- **Done when:** exit code 137 আর 143 এর পার্থক্য, আর app কেন `0.0.0.0` এ listen করবে ব্যাখ্যা করতে পারেন।

M0.3 Networking basics · L2

- **Purpose:** Kubernetes এর অর্ধেক সমস্যা networking সমস্যা।
- **Core concepts:** IP address, private range (`10.x`, `172.16–31.x`, `192.168.x`), CIDR (`/16`, `/24`), port, TCP connection, HTTP request/response ও status code, DNS (A record, resolver), routing, NAT, reverse proxy, load balancer (L4 vs L7)।
- **Commands:** `ip addr` / `ifconfig`, `ping`, `nslookup`, `dig`, `curl -v`, `nc -zv host port`।
- **Hands-on:** `curl -v` দিয়ে একটা request এর DNS lookup, TCP connect, TLS, HTTP response ধাপগুলো পড়া।
- **Common mistakes:** "connection refused" (port এ কেউ listen করছে না) আর "timeout" (packet পৌঁছাচ্ছে না/block হচ্ছে) গুলিয়ে ফেলা।
- **Production relevance:** Service, DNS, NetworkPolicy, Gateway API debug করতে সরাসরি লাগে।
- **Done when:** একটা browser request কীভাবে DNS → TCP → HTTP → load balancer → server যায়, এঁকে ব্যাখ্যা করতে পারেন।

M0.4 Servers, VMs, containers, images, registries · L2 · [DOCKER-MIN]

- **Purpose:** Node, Pod, container — এই শব্দগুলোর বাস্তব রূপ বোঝা।
- **Core concepts:** physical server vs VM (hypervisor) vs container (shared kernel, namespaces + cgroups দিয়ে isolation); image = read-only template, layer; container = image এর চলমান instance; registry (Docker Hub, GHCR, ECR); tag vs digest; `latest` tag এর বিপদ; container ephemeral।
- **Internals (recognize only):** Linux namespaces (PID, network, mount) আর cgroups (CPU/memory limit) — এগুলোই পরে resource limit হয়ে ফিরে আসবে।
- **Hands-on:** [DOCKER-TRACK] এ একটা public image চালিয়ে, container মুছে, ভেতরের data হারানো দেখা।
- **Common mistakes:** container কে "ছোট VM" ভাবা; `latest` tag production এ ব্যবহার।
- **Done when:** VM আর container এর পার্থক্য, আর image tag বনাম digest কেন গুরুত্বপূর্ণ বলতে পারেন।

M0.5 YAML ও basic shell scripting · L2

- **Purpose:** প্রতিটা Kubernetes manifest YAML এ লেখা; একটা indentation ভুলে পুরো deploy fail।
- **Core concepts:** map, list, nested structure, indentation (space, tab না), multi-line string (`|` , `>`), multiple documents (`---`), quoting (`"yes"` , `"0123"` , port string vs number), anchors (recognize)। Shell: variable, `for` loop, `if` , script এর exit code।
- **Hands-on:** ভাঙা YAML ঠিক করা; `yq` দিয়ে YAML থেকে value বের করা।
- **Common mistakes:** `on: yes` জাতীয় value boolean হয়ে যাওয়া; list আর map গুলিয়ে ফেলা।
- **Done when:** একটা nested YAML হাতে লিখে, তার JSON রূপ মুখে বলতে পারেন।

M0.6 Git, build artifacts ও deployment concepts · L2

- **Purpose:** Kubernetes এ যা deploy হয় তা একটা build artifact (image); কোন commit থেকে কোন image — traceability জরুরি।
- **Core concepts:** source → build → artifact → release → deploy; immutable artifact; environment-specific config আলাদা রাখা (12-factor: config in env, stateless process, logs to stdout, disposability)।
- **Done when:** কেন image এ secret/config bake করা উচিত না, আর কেন image tag এ git SHA দেওয়া ভালো, ব্যাখ্যা করতে পারেন।

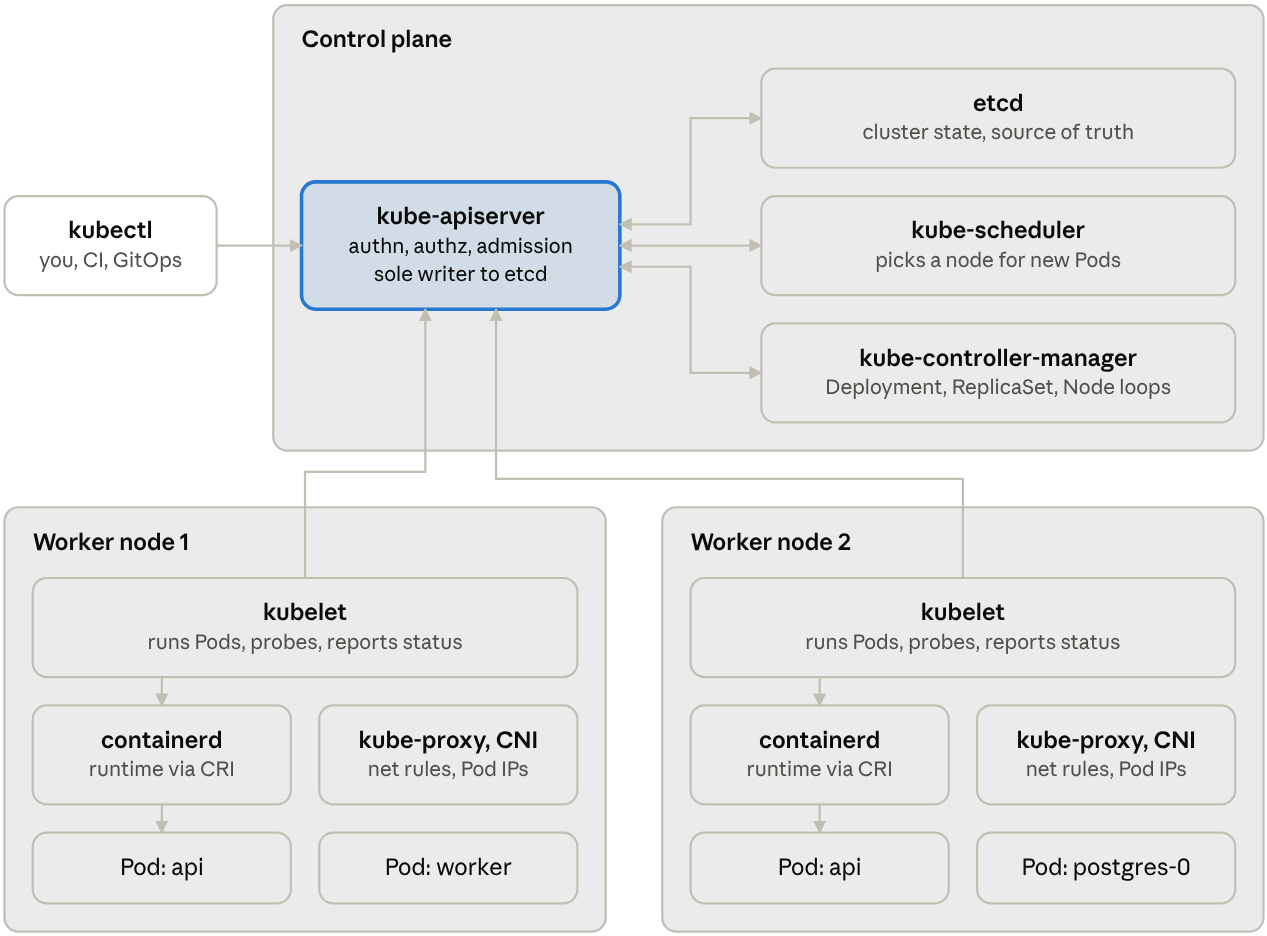
Phase 1 — Kubernetes Fundamentals

M1.1 Kubernetes কেন · L3

- **Purpose:** একটা server এ `docker run` করলেই হয় — তাহলে Kubernetes কেন? ১০০টা container, ১০টা server, crash, scaling, deploy — এই orchestration problem টা বোঝা।
- **Core concepts:** imperative ("এটা করো") vs declarative ("এমন অবস্থা চাই"); desired state vs actual state; self-healing; bin packing; Kubernetes কী **না** (PaaS না, build tool না, database manager না)।
- **Analogy:** Kubernetes একটা thermostat এর মতো — আপনি 24°C বলে দেন, সে বারবার মেপে ঠিক করে।
- **Production relevance:** কখন Kubernetes overkill (ছোট app, ছোট team → managed PaaS ভালো হতে পারে) সেটা বলতে পারাও senior skill।
- **Done when:** desired state reconciliation এক মিনিটে team কে বোঝাতে পারেন, আর Kubernetes কখন ব্যবহার করবেন না তা বলতে পারেন।

M1.2 Architecture: control plane ও worker nodes · L3

- **Purpose:** প্রতিটা failure বুঝতে হলে কোন component কী করে জানতে হবে।
- **Control plane: kube-apiserver** (একমাত্র দরজা; সব component তার সাথেই কথা বলে, authentication/authorization/admission এখানে), **etcd** (cluster এর সব state রাখা key-value database; একমাত্র source of truth), **kube-scheduler** (নতুন Pod কোন node এ যাবে ঠিক করে), **kube-controller-manager** (Deployment, ReplicaSet, Node, Job ইত্যাদি controller loop চালায়), **cloud-controller-manager** (cloud load balancer, node lifecycle)।
- **Worker node: kubelet** (node এর agent; API server থেকে নিজের Pod দেখে container runtime কে চালাতে বলে, probe চালায়, status report করে), **container runtime** (containerd বা CRI-O, CRI interface দিয়ে), **kube-proxy** (Service IP কে Pod IP তে translate করার rule বসায়; কিছু cluster এ Cilium এর মতো eBPF alternative), **CNI plugin** (Pod কে IP দেয় ও network জোড়ে), **CoreDNS** (cluster DNS, add-on হিসেবে)।
- **বুঝতে হবে vs চিনলেই চলবে:** API server, etcd এর ভূমিকা, scheduler, controller, kubelet → বুঝতে হবে (L3)। etcd Raft consensus, API server watch cache, CRI gRPC details → চিনলেই চলবে (L1)।
- **Commands:** `kubectl get nodes -o wide`, `kubectl get pods -n kube-system`, `kubectl cluster-info`, `kubectl get --raw /readyz?verbose`।
- **Hands-on:** kind cluster এ `kube-system` namespace এর প্রতিটা Pod চিনে কোনটা কোন component লেখা।
- **Done when:** খালি কাগজে control plane + ২টা node এর diagram এঁকে প্রতিটা arrow ব্যাখ্যা করতে পারেন।



Kubernetes architecture · control plane + 2 worker nodes

kubectl, scheduler, controller আর প্রতিটা node এর kubelet — সবাই শুধু API server কে watch ও update করে; etcd তে পড়া-লেখা করে একমাত্র API server। এই কারণেই API server down হলে চলমান Pod চলতে থাকে, কিন্তু নতুন কিছু হয় না।

M1.3 Reconciliation loop ও request flow · L3

- **Purpose:** Kubernetes এর "মস্তিষ্ক" — সব কিছু watch → compare → act loop।
- **`kubectl apply -f deploy.yaml` এর পর কী হয়:**
  1. kubectl kubeconfig পড়ে API server এ HTTPS request পাঠায়।
  2. API server authenticate → authorize (RBAC) → admission (mutating, তারপর validating) → schema validation করে, তারপর etcd তে Deployment লেখে।
  3. Deployment controller watch করে নতুন Deployment দেখে একটা ReplicaSet বানায়।
  4. ReplicaSet controller দেখে ৩টা Pod দরকার, ০টা আছে → ৩টা Pod object বানায় (এখনো কোনো node নেই, status Pending)।

5. Scheduler unscheduled Pod দেখে, filter (resource আছে? taint?) ও score করে node বেছে `nodeName` bind করে।
6. সেই node এর kubelet দেখে তার নামে নতুন Pod → CNI দিয়ে network, runtime দিয়ে image pull, container start।
7. kubelet probe চালিয়ে Pod Ready report করে → EndpointSlice controller Pod IP কে Service এর endpoint এ যোগ করে → kube-proxy rule update।

- **Pod crash করলে:** container exit → kubelet `restartPolicy` অনুযায়ী restart করে, বারবার হলে exponential backoff (CrashLoopBackOff)। এটা kubelet এর কাজ, controller এর না — Pod একই node এ থাকে।
- **Node unavailable হলে:** kubelet heartbeat (Lease) বন্ধ → node controller node কে `NotReady` করে → taint বসে → default ~৫ মিনিট পর ওই node এর Pod evict হয় → ReplicaSet controller দেখে replica কম → নতুন Pod বানায় → scheduler অন্য node এ পাঠায়। (Bare Pod হলে কেউ আবার বানায় না — তাই সরাসরি Pod চালাবেন না।)
- **Hands-on:** `kubectl get events -w` চালিয়ে রেখে Deployment apply করে ধাপগুলো event এ মেলানো; kind এর একটা worker node container `docker stop` করে Pod অন্য node এ সরতে দেখা [LAB-SAFE]।
- **Done when:** উপরের ৭ ধাপ, crash flow ও node failure flow নিজের ভাষায় বলতে পারেন।

M1.4 Local cluster setup · L2

- **Options তুলনা:**

Tool	কীভাবে চলে	ভালো দিক	সীমাবদ্ধতা
kind	প্রতিটা node একটা Docker container	হালকা, multi-node সহজ, CI তে জনপ্রিয়, upstream Kubernetes	LoadBalancer/ingress এর জন্য extra setup
minikube	VM বা container এ	অনেক addon (metrics-server, dashboard) এক command এ	একটু ভারী
k3d	k3s (হালকা distro) Docker এ	খুব দ্রুত, ছোট	k3s কিছু component বদলায় (upstream থেকে একটু আলাদা)

Tool	কীভাবে চলে	ভালো দিক	সীমাবদ্ধতা
Docker Desktop / Rancher Desktop	built-in single node	এক click	multi-node নেই, node failure practice করা যায় না

- **সিদ্ধান্ত:** আমরা **kind** ব্যবহার করব — multi-node (১ control plane + ২ worker) বানিয়ে scheduling ও node failure practice করা যায়, আর upstream Kubernetes এর কাছাকাছি। minikube backup option।
- **Hands-on:** kind config file দিয়ে ৩-node cluster; `kubectl` install; version skew (kubectl ±১ minor) চেক।
- **Common mistakes:** Docker এ যথেষ্ট RAM না দেওয়া; ভুল context এ command চালানো।
- **Done when:** cluster delete করে ২ মিনিটে আবার বানাতে পারেন।

M1.5 kubectl essentials, kubeconfig, contexts · L3

- **Core concepts:** kubeconfig (cluster + user + context), current context, namespace default; output formats (`-o wide/yaml/json/jsonpath`), `--dry-run=client -o yaml` দিয়ে YAML generate।
- **Commands:** `get` , `describe` , `logs` (`-f` , `--previous` , `-c`), `exec` , `apply` , `diff` , `delete` , `explain` , `port-forward` , `top` , `rollout` , `config get-contexts` , `config use-context` , `api-resources` ।
- **Safety:** `kubectl delete` , `scale --replicas=0` , `drain` , `edit` → production এ [PROD-CAUTION]; সবসময় আগে `kubectl config current-context` দেখুন। Production এ `kubectl edit` না করে Git এ বদলে apply/GitOps।
- **Hands-on:** shell prompt এ current context দেখানো (kube-ps1); `kubectl explain deployment.spec.strategy` দিয়ে docs ছাড়া field খোঁজা।
- **Done when:** কোনো tutorial না দেখে একটা Pod এর log, YAML, event, IP বের করতে পারেন।

M1.6 API objects ও manifest anatomy · L3

- **Core concepts:** `apiVersion` (group/version, যেমন `apps/v1`), `kind` , `metadata` (name, namespace, labels, annotations), `spec` (আপনি যা চান), `status` (Kubernetes যা দেখছে; আপনি লেখেন না); API groups ও versions (alpha/beta/GA); imperative command vs declarative `apply` ; server-side apply (recognize)।
- **Hands-on:** একটা Pod YAML শূন্য থেকে হাতে লেখা; `kubectl get pod x -o yaml` থেকে কোন field আপনার আর কোনটা system এর আলাদা করা।

- **Common mistakes:** `status` field সহ YAML copy করে apply করা; deprecated `apiVersion` ব্যবহার।
- **Done when:** যেকোনো manifest দেখে spec আর status এর ভূমিকা ব্যাখ্যা করতে পারেন।

Phase 2 — Core Workloads

M2.1 Pods · L3

- **Purpose:** Kubernetes এর সবচেয়ে ছোট deploy unit — এক বা একাধিক container যারা network ও storage share করে।
- **Core concepts:** Pod phase (Pending, Running, Succeeded, Failed, Unknown) vs container state (Waiting, Running, Terminated) — আলাদা জিনিস; `restartPolicy`; একই Pod এর container রা `localhost` দিয়ে কথা বলে; init container (আগে চলে শেষ হয়, যেমন DB ready কিনা চেক); native sidecar container (`restartPolicy: Always` সহ init container; v1.33 এ GA) [version-sensitive]; Pod ephemeral, IP বদলায়।
- **Internals:** pause/sandbox container network namespace ধরে রাখে (recognize)।
- **Hands-on:** Node.js Pod চালানো; init container দিয়ে Postgres ready হওয়ার অপেক্ষা; sidecar দিয়ে log tail।
- **Common mistakes:** সরাসরি Pod deploy করা (মরে গেলে কেউ ফেরত আনে না); এক Pod এ app + database রাখা।
- **Troubleshooting:** `describe` এর Events, `logs --previous`, container state এর `reason`।
- **Done when:** কখন multi-container Pod যুক্তিসঙ্গত আর কখন আলাদা Deployment, উদাহরণ সহ বলতে পারেন।

M2.2 ReplicaSets ও Deployments · L3 · **Project 2 এর আগে [DOCKER-TRACK] Dockerfile শেষ করুন**

- **Purpose:** N টা identical Pod সবসময় চালু রাখা আর version বদলানো।
- **Core concepts:** ReplicaSet = নির্দিষ্ট সংখ্যক Pod নিশ্চিত করে; Deployment = ReplicaSet এর উপর version management; `spec.selector` immutable; `pod-template-hash` label; ownerReferences ও garbage collection।
- **Hands-on:** SubTrack API কে ৩ replica Deployment এ চালানো; একটা Pod delete করে নতুন আসা দেখা; ReplicaSet সরাসরি edit করলে কী হয় দেখা।
- **Common mistakes:** selector আর template label না মেলা (apply reject হয়); একই label এ দুই Deployment।
- **Done when:** Deployment → ReplicaSet → Pod ownership chain `kubectl` দিয়ে দেখাতে পারেন।

M2.3 Labels, selectors, annotations · L3

- **Purpose:** Kubernetes এ সব জোড়া লাগে label দিয়ে — Service কোন Pod এ traffic দেবে, Deployment কোন Pod এর মালিক, NetworkPolicy কাকে block করবে।
- **Core concepts:** label (selection এর জন্য) vs annotation (metadata/tool config এর জন্য, select করা যায় না); equality ও set-based selector; recommended labels (`app.kubernetes.io/name`, `/version`, `/component`)।
- **Commands:** `get pods -l app=api,tier!=db`, `--show-labels`, `label`, `annotate`।
- **Debugging exercise:** Service এ ভুল selector দিয়ে "Service আছে কিন্তু কোনো endpoint নেই" সমস্যা বানানো ও ঠিক করা।
- **Done when:** label দিয়ে একটা Pod কে Deployment থেকে "বিচ্ছিন্ন" করে debug এর জন্য রেখে দিতে পারেন (ReplicaSet নতুন Pod আনবে)।

M2.4 Namespaces · L2

- **Purpose:** একটা cluster কে logical ভাগে ভাগ করা (team, environment, app)।
- **Core concepts:** namespaced vs cluster-scoped resource; default namespaces (`default`, `kube-system`, `kube-public`, `kube-node-lease`); namespace নিজে security boundary না (RBAC, NetworkPolicy, quota লাগে); namespace delete করলে ভেতরের সব মুছে যায় [PROD-CAUTION]।
- **Done when:** কোন resource namespaced তা `kubectl api-resources --namespaced` দিয়ে বের করতে পারেন।

M2.5 Rolling updates, rollout history, rollback · L3

- **Core concepts:** `RollingUpdate` (`maxSurge`, `maxUnavailable`) vs `Recreate`; নতুন ReplicaSet scale up, পুরনোটা down; readiness probe ছাড়া rolling update অন্ধ; `revisionHistoryLimit`; `progressDeadlineSeconds`; rollback = পুরনো ReplicaSet এর template ফেরত আনা (data/DB migration ফেরত যায় না!)।
- **Commands:** `rollout status`, `rollout history`, `rollout undo --to-revision`, `rollout pause/resume`, `rollout restart`।
- **Hands-on:** v1 → v2 update চলার সময় `curl` loop চালিয়ে error হয় কিনা দেখা; ইচ্ছা করে ভাঙা image দিয়ে stuck rollout বানিয়ে rollback।
- **Production relevance:** DB schema change এর সাথে rollback এর সম্পর্ক (expand-contract pattern) — বাস্তবে সবচেয়ে বেশি ঝামেলা এখানেই।
- **Done when:** একটা failed rollout detect, diagnose ও rollback করতে পারেন, আর কেন rollback সবসময় নিরাপদ না ব্যাখ্যা করতে পারেন।

Phase 3 — Networking ও Configuration

M3.1 Services · L3

- **Purpose:** Pod এর IP বারবার বদলায়; Service একটা স্থায়ী virtual IP ও DNS name দেয় আর traffic ready Pod গুলোর মধ্যে ভাগ করে।
- **Core concepts:** ClusterIP (default, শুধু cluster এর ভেতরে), NodePort (প্রতিটা node এর একটা port), LoadBalancer (cloud LB বানায়; local এ cloud-provider-kind বা MetalLB লাগে), headless (`clusterIP: None`, সরাসরি Pod IP — StatefulSet এর জন্য), ExternalName (DNS alias); `port` vs `targetPort` vs `nodePort`; selector → EndpointSlice; শুধু Ready Pod endpoint এ থাকে; session affinity (recognize)।
- **Internals:** Service IP আসলে কোনো interface এ নেই — kube-proxy প্রতিটা node এ iptables/IPVS/nftables rule বসায় যা packet এর destination বদলে Pod IP করে দেয়।
- **Commands:** `get svc`, `get endpointslices -l kubernetes.io/service-name=api`, `port-forward svc/api 8080:80`, `run tmp --rm -it --image=busybox -- wget -qO- http://api`।
- **Hands-on:** SubTrack API এর জন্য ClusterIP Service; একটা temporary Pod থেকে call; NodePort দিয়ে বাইরে থেকে access।
- **Common mistakes:** `targetPort` ভুল (container যে port এ listen করে সেটা নয়); selector typo → empty endpoints; readiness fail করায় endpoint খালি।
- **Troubleshooting workflow:** Service আছে? → endpoints আছে? → না থাকলে selector vs Pod label মেলান → Pod Ready? → `targetPort` এ app listen করছে? → Pod থেকে সরাসরি Pod IP তে call হয়?
- **Done when:** "Service কাজ করছে না" সমস্যা ৫ ধাপে isolate করতে পারেন।

M3.2 DNS ও service discovery · L3

- **Core concepts:** CoreDNS; record format `<svc>.<ns>.svc.cluster.local`; একই namespace এ শুধু `<svc>`; Pod এর `/etc/resolv.conf` (search domain, `ndots:5`); headless Service এ প্রতিটা Pod এর নিজের record (`postgres-0.postgres`); ExternalName।
- **Hands-on:** `nslookup api`, `nslookup api.other-ns`, ভুল namespace থেকে short name fail হওয়া দেখা।
- **Common mistakes:** অন্য namespace এর Service কে শুধু নামে ডাকা; CoreDNS নিজে down থাকলে সব Service "ভাঙা" মনে হওয়া।
- **Troubleshooting:** debug Pod থেকে `nslookup kubernetes.default` → fail হলে `kubectl -n kube-system get pods -l k8s-app=kube-dns` ও CoreDNS logs।
- **Done when:** DNS failure আর Service/endpoint failure আলাদা করতে পারেন।

M3.3 Cluster networking model · L2 (internals L1)

- **Core concepts:** Kubernetes এর networking নিয়ম — প্রতিটা Pod এর নিজস্ব IP, সব Pod NAT ছাড়া একে অপরের সাথে কথা বলতে পারে; CNI plugin এই নিয়ম বাস্তবায়ন করে (kind এ kindnet; production এ Calico, Cilium, cloud VPC CNI); Pod CIDR vs Service CIDR vs node network; kube-proxy modes: iptables, IPVS, nftables (v1.33 এ GA) [version-sensitive]; eBPF-based replacement (Cilium) — recognize।
- **Hands-on:** তিনটা IP range (node, Pod, Service) চিহ্নিত করা; ভিন্ন node এর দুই Pod এর মধ্যে ping।
- **Postpone:** CNI plugin internals, overlay vs routed, eBPF → M12.7 (L4)।
- **Done when:** Pod A (node 1) থেকে Service হয়ে Pod B (node 2) পর্যন্ত packet এর পথ বলতে পারেন।

M3.4 ConfigMaps ও environment variables · L3

- **Purpose:** একই image কে dev/staging/prod এ ভিন্ন config দিয়ে চালানো।
- **Core concepts:** `env`, `envFrom`, `valueFrom.configMapKeyRef`; file হিসেবে mount; env var update হয় না যতক্ষণ Pod restart না হয়, কিন্তু mounted file কিছুক্ষণ পর update হয় (`subPath` mount হলে হয় না); immutable ConfigMap; config বদলালে rollout trigger করার pattern (checksum annotation)।
- **Hands-on:** SubTrack এর `LOG_LEVEL`, `PAYMENT_MODE=mock` ConfigMap থেকে; config বদলে Pod এ নতুন value না আসা দেখা ও `rollout restart`।
- **Common mistakes:** ConfigMap এ password রাখা; ConfigMap না থাকলে Pod `CreateContainerConfigError` এ আটকে থাকা।
- **Done when:** কোন config কখন Pod এ পৌঁছায় তা বলতে পারেন।

M3.5 Secrets · L3

- **Core concepts:** Secret এর data শুধু **base64 encoded — encrypted না**; etcd তে encryption at rest আলাদা করে চালু করতে হয়; RBAC দিয়ে `get secrets` সীমিত; Secret type (`Opaque`, `kubernetes.io/dockerconfigjson`, `kubernetes.io/tls`); `imagePullSecrets`; env var বনাম file mount (file mount একটু নিরাপদ — log/crash dump এ কম leak হয়)।
- **Hands-on:** `DATABASE_URL` Secret বানানো (`kubectl create secret generic --from-literal`); `kubectl get secret -o yaml` দেখে base64 decode করে বোঝা কেন এটা encryption না।
- **Common mistakes:** Secret YAML Git এ commit করা; Secret এর মান log করা।
- **Production relevance:** real setup এ External Secrets Operator / Sealed Secrets / cloud secret manager → M8.5।

- **Done when:** Secret কী থেকে রক্ষা করে আর কী থেকে করে না, পরিষ্কার বলতে পারেন।

Phase 4 — Storage ও Stateful Workloads

M4.1 Volumes · L2

- **Core concepts:** container filesystem ephemeral; Pod-level volume; `emptyDir` (Pod এর জীবনকাল পর্যন্ত, container restart এ থাকে, Pod delete এ যায়; `medium: Memory`); configMap/secret volume; projected volume; `hostPath` (production এ প্রায় কখনো না) [PROD-CAUTION]।
- **Hands-on:** দুই container এর মধ্যে `emptyDir` দিয়ে file share।
- **Done when:** কোন volume কতক্ষণ বাঁচে তা table আকারে বলতে পারেন।

M4.2 PersistentVolume, PVC, StorageClass, CSI · L3

- **Purpose:** Pod মরে গেলেও data রাখা।
- **Core concepts:** PV (আসল storage), PVC (storage এর অনুরোধ), StorageClass (কীভাবে dynamic ভাবে বানাবে); binding; access modes (RWO, ROX, RWX, RWOP); reclaim policy (`Delete` vs `Retain` — Delete হলে PVC মুছলে data মুছে যায়!) [PROD-CAUTION]; `volumeBindingMode: WaitForFirstConsumer` (zone সমস্যা এড়াতে); CSI driver; volume expansion; snapshots (recognize)।
- **Internals:** PVC → (dynamic) provisioner disk বানায় → PV → attach (node এ) → mount (Pod এ); cloud disk সাধারণত একটা zone এ বাঁধা।
- **Hands-on:** kind এর local-path StorageClass এ PVC; Pod delete করে data টিকে থাকা দেখা।
- **Debugging exercise:** অস্তিত্বহীন StorageClass দিয়ে PVC → Pending; RWO volume দুই node এর Pod এ mount চেষ্টা → Multi-Attach error।
- **Troubleshooting:** `describe pvc` events → StorageClass আছে? provisioner চলছে? zone/node mismatch?
- **Done when:** PVC Pending এর ৩টা কারণ ও তাদের সমাধান বলতে পারেন।

M4.3 StatefulSets · L2

- **Core concepts:** স্থির identity (`postgres-0`, `postgres-1`), ক্রমানুসারে start/stop, প্রতিটা replica এর নিজের PVC (`volumeClaimTemplates`), headless Service; StatefulSet delete করলে PVC থেকে যায় (default)।
- **Hands-on:** একটা single-replica PostgreSQL StatefulSet; Pod delete করে একই নাম ও একই data নিয়ে ফেরত আসা।
- **Common mistakes:** StatefulSet কে replication মনে করা — StatefulSet শুধু identity ও storage দেয়, Postgres replication আপনাকে (বা operator কে) করতে হবে।

- **Done when:** Deployment আর StatefulSet এর পার্থক্য ৪টা point এ বলতে পারেন।

M4.4 PostgreSQL on Kubernetes vs managed database · L2

- **Core concepts:** Kubernetes এ DB চালানোর trade-off (backup, failover, upgrade, tuning আপনার দায়িত্ব) vs managed DB (RDS, Cloud SQL); মাঝামাঝি পথ: Postgres operator (যেমন CloudNativePG) — recognize; app থেকে external DB কে ExternalName/Service দিয়ে abstract করা।
- **Production relevance:** বেশিরভাগ ছোট-মাঝারি team production এ managed DB নেয়, Kubernetes এ রাখে stateless app। Lab এ আমরা Kubernetes এ Postgres চালাব শেখার জন্য।
- **Done when:** আপনার team এর জন্য কোনটা বেছে নেবেন, যুক্তিসহ ১ পাতা লিখতে পারেন।

Phase 5 — Health, Resources ও Scheduling

M5.1 Probes · L3

- **Core concepts: liveness** (fail → kubelet container restart করে), **readiness** (fail → Service endpoint থেকে বাদ, restart না), **startup** (ধীরে start হওয়া app এর জন্য; পাস না হওয়া পর্যন্ত বাকি দুটো বন্ধ); HTTP/TCP/exec/gRPC probe; `initialDelaySeconds`, `periodSeconds`, `failureThreshold`, `timeoutSeconds`।
- **Hands-on:** Node.js এ `/healthz` (process বেঁচে আছে) আর `/readyz` (DB connection আছে) আলাদা endpoint।
- **Common mistakes (খুব গুরুত্বপূর্ণ):** liveness probe এ DB check দেওয়া → DB একটু slow হলেই সব Pod restart → cascading outage; timeout খুব কম; readiness না থাকায় rolling update এ 502।
- **Done when:** কোন check কোন probe এ যাবে, আর ভুল জায়গায় দিলে কী outage হয় বলতে পারেন।

M5.2 Requests, limits, QoS, OOMKilled, throttling · L3

- **Core concepts:** request = scheduler এর হিসাব (guaranteed reservation); limit = সর্বোচ্চ; memory limit পেরোলে OOMKilled (exit 137); CPU limit পেরোলে kill না, throttle (app slow); QoS class: Guaranteed, Burstable, BestEffort — node এ memory চাপ পড়লে কে আগে evict হয়; Node.js heap (`--max-old-space-size`) আর container memory limit মেলানো।
- **Internals:** কোনো Pod schedule হবে কিনা ঠিক হয় request এর যোগফলে, আসল usage এ না; limit বাস্তবায়ন হয় cgroups দিয়ে।
- **Hands-on:** memory leak endpoint বানিয়ে OOMKilled দেখা; CPU-heavy endpoint এ throttling latency দেখা; খুব বড় request দিয়ে Pod Pending।
- **Common mistakes:** request না দেওয়া (scheduler অন্ধ); CPU limit খুব কম দিয়ে latency spike (অনেক team CPU limit দেয় না, শুধু request দেয় — trade-off আলোচনা করব)।

- **Done when:** OOMKilled, Pending (Insufficient cpu), throttling — তিনটা আলাদা লক্ষণ চিনে কারণ বলতে পারেন।

M5.3 LimitRange ও ResourceQuota · L2

- **Core concepts:** namespace এ default request/limit (LimitRange), মোট সীমা (ResourceQuota); quota থাকলে request ছাড়া Pod reject হয়।
- **Done when:** একটা namespace কে ২ CPU / 4Gi এ সীমিত করে তার প্রভাব দেখাতে পারেন।

M5.4 Scheduling · L2

- **Core concepts:** scheduler এর filter → score; `nodeSelector`; node affinity (required vs preferred); Pod affinity/anti-affinity (একই app এর replica ভিন্ন node এ); taints & tolerations (node বলে "আমার কাছে এসো না", toleration বলে "আমি সহ্য করব" — toleration নিজে Pod কে আকর্ষণ করে না); `topologySpreadConstraints` (zone/node জুড়ে সমান ভাগ); priority classes (recognize)।
- **Hands-on:** kind এর একটা node taint করে Pod এর আচরণ দেখা; anti-affinity দিয়ে ৩ replica তিন node এ।
- **Debugging exercise:** অসম্ভব affinity দিয়ে Pending Pod, `describe` এর scheduler message পড়া (`0/3 nodes are available: ...`)।
- **Done when:** scheduler এর Pending message পড়ে কারণ বলতে পারেন।

M5.5 Scaling ও self-healing · L2

- **Core concepts:** `kubectl scale`; HorizontalPodAutoscaler (`autoscaling/v2`) — CPU/memory utilization **request এর শতাংশ হিসেবে** মাপে, তাই request ছাড়া HPA কাজ করে না; metrics-server; scale-up/down behavior ও stabilization window; HPA আর manual `replicas` এর সংঘাত (GitOps এ)।
- **Hands-on:** metrics-server install; load generator দিয়ে SubTrack API কে ২ থেকে ৬ replica তে যেতে দেখা।
- **Done when:** HPA কেন scale করছে না, তার ৩টা কারণ বলতে পারেন (metrics-server নেই, request নেই, maxReplicas)।

M5.6 Availability: PDB, graceful shutdown · L3

- **Core concepts:** Pod termination flow: Pod `Terminating` হয় → endpoint থেকে সরানো শুরু **এবং একই সাথে** preStop hook → SIGTERM → `terminationGracePeriodSeconds` (default 30s) → SIGKILL; এই race এর কারণে Node.js এ SIGTERM পেয়ে নতুন request নেওয়া বন্ধ + চলমান request শেষ করা দরকার, আর ছোট preStop sleep; PodDisruptionBudget (node drain/upgrade এর সময় কমপক্ষে কতগুলো Pod থাকবে) — voluntary vs involuntary disruption।

- **Hands-on:** rolling update এর সময় load test চালিয়ে graceful shutdown ছাড়া আর সহ error rate তুলনা; PDB সহ `kubectl drain` [LAB-SAFE, production এ PROD-CAUTION]।
- **Done when:** zero-downtime deploy এর জন্য দরকারি ৪টা জিনিস (readiness, graceful shutdown, preStop, PDB) ব্যাখ্যা ও প্রমাণ করতে পারেন।

Phase 6 — External Traffic Management

M6.1 Ingress · L2

- **Purpose:** একটা IP/domain দিয়ে অনেক Service এ HTTP traffic পাঠানো (host/path based routing)।
- **Core concepts:** Ingress resource শুধু নিয়ম; কাজ করে একটা Ingress **controller**; `ingressClassName`; annotation-নির্ভর config (controller ভেদে আলাদা — portability সমস্যা)।
- **বর্তমান status [version-sensitive]:** Ingress API এখনো supported কিন্তু frozen; জনপ্রিয় **ingress-nginx controller March 2026 এ retire হয়েছে** (চলমান deployment ভাঙবে না, কিন্তু আর security fix আসবে না)। পুরনো tutorial এর `ingress-nginx` install অনুসরণ করবেন না।
- **কেন তবু শিখবেন:** বিদ্যমান অনেক cluster এ Ingress আছে; পড়তে ও migrate করতে পারতে হবে।
- **Done when:** একটা Ingress YAML পড়ে এর সমতুল্য HTTPRoute লিখতে পারেন।

M6.2 Gateway API · L2

- **Purpose:** Ingress এর আধুনিক, role-oriented উত্তরসূরি।
- **Core concepts: GatewayClass** (কোন implementation — infra provider এর), **Gateway** (listener: port, protocol, hostname, TLS — cluster operator এর), **HTTPRoute** (path/header match, backend, weight — app developer এর); GRPCRoute; cross-namespace route এর জন্য ReferenceGrant; traffic splitting (canary weight); Gateway API CRD আলাদা install হয়, তার version Kubernetes version থেকে আলাদা।
- **Implementations:** Envoy Gateway, Cilium, Traefik, NGINX Gateway Fabric, Istio, kgateway, cloud-native (GKE Gateway ইত্যাদি)। Lab এ আমরা একটা conformant implementation (সম্ভবত Envoy Gateway) বেছে নেব module শুরুর সময় current status দেখে।
- **Hands-on:** `api.subtrack.local` → API Service, `/` → frontend; v1/v2 এ 90/10 weight split।
- **Troubleshooting:** `kubectl describe gateway` / `httproute` এর `status.conditions` (Accepted, ResolvedRefs, Programmed) — এখানেই বেশিরভাগ ভুল দেখা যায়।
- **Done when:** route কেন traffic পাচ্ছে না, status condition পড়ে বলতে পারেন।

M6.3 TLS ও cert-manager · L2

- **Core concepts:** TLS certificate, CA, `kubernetes.io/tls` Secret; cert-manager (Issuer/ClusterIssuer, Certificate); ACME/Let's Encrypt (HTTP-01, DNS-01) — local এ self-signed CA।
- **Hands-on:** self-signed ClusterIssuer দিয়ে Gateway listener এ HTTPS।
- **Common mistakes:** cert expire হওয়া monitor না করা; staging vs production ACME গুলিয়ে rate limit খাওয়া।
- **Done when:** একটা certificate কীভাবে issue হয়ে Gateway এ লাগে, flow বলতে পারেন।

Phase 7 — Specialized Workloads

M7.1 Jobs · L2

- **Core concepts:** কাজ শেষ হলে থামে এমন Pod; `completions`, `parallelism`, `backoffLimit`, `activeDeadlineSeconds`, `ttlSecondsAfterFinished`; `restartPolicy: OnFailure/Never`; indexed jobs (recognize)।
- **Pattern:** DB migration Job deploy এর আগে (Helm hook বা Argo CD sync wave); migration idempotent হতে হবে।
- **Hands-on:** SubTrack এর migration Job; ইচ্ছা করে fail করিয়ে backoff দেখা।
- **Done when:** কেন migration app container এর startup এ না চালিয়ে আলাদা Job এ চালানো ভালো, বলতে পারেন।

M7.2 CronJobs · L2

- **Core concepts:** cron schedule, `timeZone` field, `concurrencyPolicy` (Allow/Forbid/Replace), `startingDeadlineSeconds`, history limits; একই সময়ে দুইবার চলার সম্ভাবনা → job idempotent হতে হবে।
- **Hands-on:** SubTrack এর "daily subscription renewal" CronJob (mock provider কে call করে); `kubectl create job --from=cronjob/renewal manual-1` দিয়ে হাতে চালানো।
- **Common mistakes:** UTC vs Asia/Dhaka সময় গুলিয়ে ফেলা; আগের run শেষ না হতেই পরেরটা শুরু।
- **Done when:** double charge এড়াতে CronJob ও app logic কীভাবে design করবেন বলতে পারেন।

M7.3 DaemonSets · L1

- **Core concepts:** প্রতিটা (বা নির্বাচিত) node এ একটা করে Pod — log collector, monitoring agent, CNI। App developer হিসেবে সাধারণত লিখবেন না, চিনবেন।
- **Done when:** `kube-system` এ কোন কোন DaemonSet আছে আর কেন, বলতে পারেন।

Phase 8 — Security

M8.1 Authentication, authorization ও RBAC · L2

- **Core concepts:** request flow: authN (কে? — certificate, OIDC token, ServiceAccount token) → authZ (পারবে? — RBAC) → admission; Kubernetes এ "User" object নেই; Role/ClusterRole (verbs × resources), RoleBinding/ClusterRoleBinding; least privilege; `cluster-admin` এর বিপদ।
- **Commands:** `kubectl auth can-i list secrets --as=system:serviceaccount:app:api -n app`, `auth whoami`।
- **Hands-on:** developer এর জন্য read-only Role; CI এর জন্য শুধু একটা namespace এ deploy করার Role।
- **Done when:** একটা `Forbidden` error পড়ে কোন Role/Binding লাগবে লিখতে পারেন।

M8.2 ServiceAccounts ও workload identity · L2

- **Core concepts:** Pod এর identity; projected, short-lived token (পুরনো long-lived Secret token deprecated pattern); `automountServiceAccountToken: false` যখন API access লাগে না; cloud workload identity (IRSA/EKS Pod Identity, GKE Workload Identity) — static cloud key এর বদলে।
- **Done when:** কেন app Pod এ default ServiceAccount token mount বন্ধ রাখা ভালো, বলতে পারেন।

M8.3 securityContext ও Pod Security Standards · L2 · **আগে [DOCKER-TRACK] non-root image শিখুন**

- **Core concepts:** `runAsNonRoot`, `runAsUser`, `readOnlyRootFilesystem`, `allowPrivilegeEscalation: false`, `capabilities.drop: [ALL]`, `seccompProfile: RuntimeDefault`; Pod Security Standards (Privileged, Baseline, Restricted) আর namespace label দিয়ে Pod Security Admission; PodSecurityPolicy v1.25 এ removed — পুরনো tutorial এ দেখলে এড়িয়ে যান।
- **Hands-on:** SubTrack namespace এ `restricted` enforce করে API কে পাস করানো (writable `/tmp` এর জন্য emptyDir লাগবে)।
- **Done when:** restricted profile এ কোনো Pod reject হলে error পড়ে ঠিক করতে পারেন।

M8.4 NetworkPolicies · L2

- **Core concepts:** default এ সব Pod সবার সাথে কথা বলতে পারে; NetworkPolicy additive allow-list; podSelector, namespaceSelector, ingress/egress; default-deny pattern; egress deny করলে DNS (port 53) allow করতে ভুলবেন না; CNI support না করলে policy চুপচাপ কিছুই করে না (kind এর default CNI তে সীমিত — lab এ Calico/Cilium লাগবে)।

- **Hands-on:** শুধু API Pod ই Postgres এ যেতে পারবে; worker কে শুধু API তে।
- **Done when:** default-deny দিয়ে শুরু করে প্রয়োজনীয় traffic খুলতে পারেন এবং DNS ভাঙার সমস্যা চেনেন।

M8.5 Production secrets · L2

- **Core concepts:** etcd encryption at rest (managed cluster এ সাধারণত চালু বা option); External Secrets Operator (AWS Secrets Manager/Vault থেকে sync); Sealed Secrets (Git এ encrypted রাখা); SOPS (recognize); rotation।
- **Done when:** GitOps এ secret কোথায় থাকবে, দুটো approach তুলনা করে বলতে পারেন।

M8.6 Supply-chain security · L1

- **Core concepts:** image vulnerability scanning (Trivy), minimal base image, image signing ও verification (cosign/Sigstore), SBOM, digest pinning।
- **Done when:** CI তে কোথায় scan বসাবেন বলতে পারেন।

Phase 9 — Packaging ও Delivery

M9.1 Kustomize · L2

- **Core concepts:** `kubectl apply -k`; base + overlays (dev/staging/prod); patches; configMapGenerator (hash suffix → auto rollout); template ছাড়া YAML customize।
- **Hands-on:** SubTrack এর base আর dev/prod overlay (replica, resource, image tag ভিন্ন)।
- **Done when:** Kustomize আর Helm কখন কোনটা, বলতে পারেন।

M9.2 Helm: chart ব্যবহার · L2 · [version-sensitive]

- **Core concepts:** chart, release, values, repository/OCI registry; `install`, `upgrade --install`, `rollback`, `uninstall`, `template`, `get values`, `history`; values override এর ক্রম। Helm 4 late 2025 এ release হয়েছে আর অনেক docs এখনো Helm 3 এর — module শুরুর সময় current version মিলিয়ে নেব।
- **Hands-on:** Helm দিয়ে cert-manager, kube-prometheus-stack install।
- **Common mistakes:** `helm upgrade` এ আগের values হারানো; chart version pin না করা।
- **Done when:** একটা third-party chart এর values পড়ে নিজের প্রয়োজনমতো configure করতে পারেন।

M9.3 Helm: নিজের chart লেখা · L2

- **Core concepts:** chart structure, Go templating, `_helpers.tpl`, `values.yaml` design, `helm lint`, hooks (pre-upgrade migration), chart testing।
- **Hands-on:** SubTrack এর জন্য একটা chart (api, worker, cronjob, migration hook)।

- **Done when:** একটা values ফাইল বদলে staging আর production render করে diff দেখাতে পারেন।

M9.4 CI/CD pipeline · L2

- **Core concepts:** CI (test → build image → scan → push, tag = git SHA) vs CD (cluster এ নতুন version); push-based CD (CI থেকে `kubectl/helm`) vs pull-based (GitOps); CI credential কে least-privilege দেওয়া; `--atomic`/rollback।
- **Hands-on:** GitHub Actions: test → build → GHCR push → kind cluster এ deploy (ephemeral kind in CI)।
- **Done when:** একটা commit থেকে কোন image, কোন cluster revision এ গেল trace করতে পারেন।

M9.5 GitOps: Argo CD · L2

- **Core concepts:** Git = desired state, cluster এ agent pull করে sync করে; drift detection; Application, sync policy, auto-sync, self-heal, prune; sync waves; image tag update flow (CI config repo তে PR করে)।
- **Hands-on:** Argo CD install, SubTrack Helm chart কে Git থেকে sync; হাতে `kubectl edit` করে drift ও self-heal দেখা।
- **Done when:** push-based CD আর GitOps এর trade-off বলতে পারেন।

M9.6 Progressive delivery · L1

- **Core concepts:** blue-green, canary, feature flag; Argo Rollouts / Flagger (recognize); Gateway API weight দিয়ে manual canary।
- **Done when:** canary কেন rolling update এর চেয়ে নিরাপদ, বলতে পারেন।

Phase 10 — Troubleshooting ও Observability

M10.1 Troubleshooting methodology ও debugging toolkit · L3

- **Purpose:** command মুখস্থ না, একটা নির্দিষ্ট ক্রমে প্রশ্ন করা — উপর থেকে নিচে (traffic) বা নিচ থেকে উপরে (Pod)।
- **Master workflow:**
  1. **Scope:** কী ভাঙা? একটা Pod, একটা Service, পুরো namespace, পুরো cluster? সর্বশেষ কী বদলেছে (deploy, config, node)?
  2. **Object state:** `kubectl get <kind> -o wide` → STATUS, READY, RESTARTS, AGE।
  3. **Events:** `kubectl describe` এর নিচের Events, আর `kubectl get events --sort-by=.lastTimestamp`।

4. **Logs:** `kubectl logs`, crash হলে `--previous`, multi-container এ `-c`।
5. **Inside:** `kubectl exec` বা shell নেই এমন image এ `kubectl debug -it pod --image=busybox --target=app` (ephemeral container)।
6. **Network path:** Pod → Pod IP → Service → DNS → Gateway, প্রতিটা স্তর আলাদা করে test।
7. **Node:** `kubectl describe node` (Conditions, Allocatable, pressure), `kubectl top`।
8. **Fix → verify → note:** একবারে একটা পরিবর্তন; ঠিক হলে কারণ লিখে রাখা (postmortem habit)।

- **Safety:** production এ debug করার সময় read-only command আগে; `delete pod` দিয়ে "ঠিক করা" প্রমাণ মুছে দেয় [PROD-CAUTION]।
- **Done when:** অজানা একটা broken namespace পেয়ে workflow অনুসরণ করে ১৫ মিনিটে root cause বলতে পারেন।

M10.2 Failure scenario lab · L3

প্রতিটা scenario আমি lab এ ভাঙব, আপনি workflow দিয়ে ঠিক করবেন।

Symptom	প্রথমে দেখুন	সাধারণ কারণ	সমাধানের দিক
CrashLoopBackOff	`logs --previous`, exit code (`describe` এ Last State)	app error, missing env/Secret, ভুল command, liveness খুব কড়া, OOM (137)	log অনুযায়ী config/code ঠিক; probe ঢিলা; memory limit
ImagePullBackOff / ErrImagePull	`describe pod` Events	ভুল image name/tag, private registry তে `imagePullSecrets` নেই, rate limit	tag ঠিক, pull secret, registry mirror
Pending Pod	`describe pod` এর scheduler message	resource কম (request বেশি), taint, affinity অসম্ভব, PVC unbound	request কমানো/node বাড়ানো, toleration, PVC ঠিক
DNS resolution fail	debug Pod থেকে `nslookup kubernetes.default`, `/etc/resolv.conf`	CoreDNS down, ভুল namespace, egress NetworkPolicy তে port 53 block	CoreDNS ঠিক, FQDN, DNS allow
Service এ traffic যায় না	`get endpointslices`, Pod labels	selector mismatch, readiness fail, ভুল `targetPort`	label/port ঠিক, readiness ঠিক
Storage failure	`describe pvc`, Pod Events (FailedMount, Multi-Attach)	StorageClass নেই, zone mismatch, RWO volume অন্য node এ attached	সঠিক StorageClass, `WaitForFirstConsumer`, পুরনো Pod শেষ হতে দেওয়া

Symptom	প্রথমে দেখুন	সাধারণ কারণ	সমাধানের দিক
Failed rollout	`rollout status`, নতুন ReplicaSet এর Pod	নতুন version crash/readiness fail, progressDeadline exceeded	`rollout undo`, কারণ ঠিক করে আবার deploy
Resource exhaustion	`top pod/node`, OOMKilled, Evicted Pod, node pressure	memory leak, limit কম, request না দেওয়া, quota শেষ	rightsizing, limit, HPA, quota
Node failure	`get nodes` (NotReady), `describe node`	kubelet বন্ধ, disk/memory pressure, network	Pod অন্য node এ সরছে কিনা দেখা, PDB/anti-affinity ঠিক ছিল কিনা

- **Done when:** ৯টার মধ্যে কমপক্ষে ৭টা hint ছাড়া সমাধান করেন।

M10.3 Logging · L2

- **Core concepts:** app stdout/stderr এ লিখবে (file এ না); node এ log rotate হয় আর Pod মুছলে log যায় → centralized logging দরকার; structured JSON log (pino), correlation/request ID; collector (Fluent Bit / OpenTelemetry Collector DaemonSet) → backend (Loki, Elasticsearch/OpenSearch, cloud logging)।
- **Hands-on:** Loki + Grafana দিয়ে SubTrack এর সব Pod এর log এক জায়গায় query।
- **Done when:** একটা request ID দিয়ে API ও worker দুই জায়গার log খুঁজে বের করতে পারেন।

M10.4 Metrics, monitoring ও alerting · L2

- **Core concepts:** metrics-server (শুধু HPA/`top` এর জন্য, history রাখে না) vs Prometheus (scrape, time series, PromQL); kube-prometheus-stack (Prometheus Operator, ServiceMonitor, Grafana, Alertmanager); kube-state-metrics; RED (Rate, Errors, Duration) ও USE method; Node.js app এ `prom-client`; alert কাকে বলে ভালো (symptom-based, actionable)।
- **Hands-on:** SubTrack এ `/metrics`, ServiceMonitor, Grafana dashboard, "5xx rate > 5% for 5m" alert।
- **Done when:** একটা dashboard থেকে latency spike এর কারণ (CPU throttling, restart, DB) খুঁজে বের করতে পারেন।

M10.5 Tracing ও OpenTelemetry · L1

- **Core concepts:** distributed trace, span; OpenTelemetry SDK ও Collector; Tempo/Jaeger।
- **Done when:** কোন সমস্যায় log/metrics যথেষ্ট না আর trace লাগে, বলতে পারেন।

Phase 11 — Production ও Cloud Operations

M11.1 Managed Kubernetes · L2

- **Core concepts:** EKS/GKE/AKS/DigitalOcean — control plane provider চালায়, আপনি node pool, networking, add-on, workloads; shared responsibility; cloud LoadBalancer Service; IAM integration; cost model।
- **Hands-on [OPTIONAL, খরচ আছে]:** free credit দিয়ে একটা ছোট managed cluster এ SubTrack deploy, শেষে সব resource delete (না হলে bill আসবে)।
- **Done when:** managed cluster এ কী আপনার দায়িত্ব আর কী provider এর, তালিকা করতে পারেন।

M11.2 High availability ও reliability · L2

- **Core concepts:** কমপক্ষে ২–৩ replica, zone জুড়ে topology spread, PDB, anti-affinity, graceful shutdown, readiness gating; single point of failure চিহ্নিত করা (একটা DB, একটা NAT); SLI/SLO/error budget basics; dependency timeout ও retry।
- **Done when:** SubTrack এর architecture দেখে ৩টা single point of failure ও সমাধান বলতে পারেন।

M11.3 Node pools ও cluster autoscaling · L1

- **Core concepts:** HPA Pod বাড়ায়, Cluster Autoscaler/Karpenter node বাড়ায় (Pending Pod দেখে); node pool (general, spot, GPU); scale-down এ PDB এর ভূমিকা।
- **Done when:** HPA আর cluster autoscaler কীভাবে একসাথে কাজ করে বলতে পারেন।

M11.4 Upgrades, version skew, deprecated APIs · L2

- **Core concepts:** release cadence (~৩টা minor/বছর, প্রতিটা ~১৪ মাস support); একবারে এক minor version; control plane আগে, তারপর node; version skew policy; removed API (যেমন পুরনো `extensions/v1beta1`) — manifest আগে migrate; `kubectl convert` ও deprecation checker tools (recognize)।
- **Done when:** একটা upgrade checklist লিখতে পারেন যাতে app manifest ও Helm chart এর API চেক থাকে।

M11.5 Backup ও disaster recovery · L2

- **Core concepts:** কী backup করবেন: (১) Git এ manifest — cluster state এর বেশিরভাগ, (২) database — সবচেয়ে জরুরি (pg_dump CronJob বা managed snapshot, PITR), (৩) PV snapshot, (৪) Velero দিয়ে cluster object ও volume; etcd snapshot (self-managed cluster এ, recognize); RPO/RTO; **backup test না করলে backup নেই**।
- **Hands-on:** pg_dump CronJob → object storage (local এ MinIO); cluster মুছে নতুন cluster এ restore।

- **Done when:** SubTrack পুরো হারিয়ে গেলে কত সময়ে কতটুকু data নিয়ে ফেরত আনবেন, বাস্তবে করে দেখাতে পারেন।

M11.6 Cost optimization · L1

- **Core concepts:** request ই খরচ (unused request = নষ্ট টাকা); rightsizing (VPA recommendation mode বা metrics দেখে); spot/preemptible node; idle environment বন্ধ; LoadBalancer ও egress খরচ; Kubecost/OpenCost (recognize)।
- **Done when:** একটা namespace এর request vs actual usage তুলনা করে সাশ্রয়ের প্রস্তাব দিতে পারেন।

M11.7 Multi-environment setup · L2

- **Core concepts:** dev/staging/prod কে আলাদা namespace না আলাদা cluster — trade-off (isolation, cost, blast radius); promotion flow (একই image digest staging থেকে prod); environment-specific overlay/values।
- **Done when:** আপনার team এর জন্য environment layout প্রস্তাব করতে পারেন।

Phase 12 — Advanced ও Specialist (optional)

এই phase এর module গুলো সব একসাথে করতে হবে না; কাজে যেটা লাগে সেটা বেছে নেবেন।

ID	Module	কখন নেবেন	Depth
M12.1	Advanced autoscaling: VPA, KEDA (queue length দিয়ে worker scale), in-place Pod resize [version-sensitive]	Queue-based worker চালালে KEDA আগে নিন	L1 (KEDA হলে L2)
M12.2	Policy as code: ValidatingAdmissionPolicy (built-in, CEL), Kyverno, OPA Gatekeeper	Team এ guardrail দরকার হলে	L1
M12.3	Terraform/OpenTofu দিয়ে cluster ও cloud resource	Infra নিজে চালালে	L1
M12.4	CRDs ও Operators (controller pattern, CloudNativePG ব্যবহার)	Operator-based DB/tool ব্যবহার করলে	L1
M12.5	Service mesh: mTLS, traffic policy (Istio ambient, Linkerd)	Many-service, zero-trust দরকার হলে	L1
M12.6	kubeadm দিয়ে cluster from scratch, certificates, etcd backup/restore	Platform engineer হতে চাইলে বা CKA দিলে	L4

ID	Module	কখন নেবেন	Depth
M12.7	Networking internals: CNI, eBPF, Cilium, kube-proxy modes	Platform/network specialist হলে	L4
M12.8	Multi-cluster, multi-tenancy, GPU/DRA scheduling	বিশেষ প্রয়োজনে	L1

Phase 13 — Capstone

M13.1 Architecture design · L3 — SubTrack এর full architecture doc: components, resources, failure modes, security model (বিস্তারিত G অংশে)।

M13.2 Build ও deploy · L3 — Helm chart + GitOps দিয়ে local cluster এ (আর ইচ্ছা হলে managed cluster এ) deploy, সব acceptance criteria পূরণ।

M13.3 Failure drills ও presentation · L3 — আমি ৫টা অজানা failure inject করব, আপনি সমাধান করে postmortem লিখবেন; শেষে team এর জন্য ২০ মিনিটের presentation।

F. Topic-level depth matrix

নিয়ম: যেটা প্রতিদিন ব্যবহার করবেন বা যেটা ভাঙলে আপনাকেই ঠিক করতে হবে → **L3**; যেটা configure করবেন কিন্তু platform team মূলত চালায় → **L2**; যেটা চিনলেই চলবে → **L1**; **L4** শুধু দুটো specialist topic এ, আর দুটোই আপনার জন্য postpone।

ID	Topic	Level	কেন এই depth	এখন postpone করা যায়
M0.1–M0.6	Linux, process, networking, container, YAML, Git basics	L2	Kubernetes বুঝতে ভিত্তি, কিন্তু sysadmin হওয়ার দরকার নেই	kernel internals, advanced shell, subnetting math
M1.1	Desired state, declarative model	L3	পুরো Kubernetes এর মূল চিন্তা; ভুল বুঝলে সব ভুল	—
M1.2	Architecture components	L3	প্রতিটা failure কোন component এ, সেটা বুঝতে লাগে	etcd Raft, API server caching
M1.3	Reconciliation ও request flow	L3	debug করার mental model	controller-runtime code

ID	Topic	Level	কেন এই depth	এখন postpone করা যায়
M1.4	Local cluster tools	L2	ব্যবহার করতে পারলেই যথেষ্ট	kind internals
M1.5	kubectl ও kubeconfig	L3	প্রতিদিনের হাতিয়ার; ভুল context = production incident	plugins (krew)
M1.6	Manifest anatomy, API versions	L3	YAML copy না করে লেখার ক্ষমতা	server-side apply field ownership details
M2.1	Pods	L3	সব workload এর মূল একক; বেশিরভাগ error এখানেই দেখা যায়	pause container, Pod sandbox
M2.2	Deployments, ReplicaSets	L3	আপনার app এভাবেই চলবে	—
M2.3	Labels, selectors	L3	ভুল selector সবচেয়ে সাধারণ "নীরব" bug	—
M2.4	Namespaces	L2	ব্যবহার সহজ, internals কম	—
M2.5	Rolling update, rollback	L3	প্রতিটা deploy এ লাগে; ভুলে downtime	—
M3.1	Services	L3	traffic না পৌঁছানোর বেশিরভাগ কারণ এখানে	IPVS/nftables rule details
M3.2	DNS	L3	"কিছু কাজ করছে না" এর অন্যতম মূল কারণ	CoreDNS plugin config
M3.3	CNI, kube-proxy model	L2	model বুঝতে হবে, implementation না	→ M12.7 (L4)
M3.4	ConfigMaps	L3	config propagation না বুঝলে রহস্যময় bug	—

ID	Topic	Level	কেন এই depth	এখন postpone করা যায়
M3.5	Secrets basics	L3	ভুল ধারণা (base64 = secure) security incident ঘটায়	KMS provider internals
M4.1	Volumes	L2	সরল, কম failure mode	—
M4.2	PV/PVC/StorageClass	L3	data loss আর stuck Pod এর উৎস	CSI driver development
M4.3	StatefulSets	L2	বেশিরভাগ team DB managed নেয়; বুঝতে হবে, গভীরে চালাতে হবে না	DB replication internals
M4.4	DB on K8s vs managed	L2	architecture সিদ্ধান্ত নিতে	Postgres operator internals
M5.1	Probes	L3	ভুল probe = cascading outage	—
M5.2	Requests/limits/QoS	L3	OOMKilled, Pending, latency — তিনটাই এখান থেকে	cgroup v2 internals
M5.3	LimitRange, Quota	L2	সাধারণত platform team বসায়, আপনি তার মধ্যে কাজ করবেন	—
M5.4	Scheduling	L2	anti-affinity, spread ব্যবহার করবেন; scheduler internals না	scheduler plugins, priority/preemption
M5.5	HPA	L2	configure করবেন; algorithm details কম লাগে	custom/external metrics → M12.1
M5.6	PDB, graceful shutdown	L3	zero-downtime এর চাবি; Node.js এ খুব সাধারণ ভুল	—

ID	Topic	Level	কেন এই depth	এখন postpone করা যায়
M6.1	Ingress	L2	পুরনো cluster পড়তে ও migrate করতে	controller-specific annotations
M6.2	Gateway API	L2	route লিখবেন ও debug করবেন; Gateway infra platform team এর	implementation internals (Envoy)
M6.3	TLS, cert-manager	L2	ব্যবহার ও debug	ACME DNS-01 details
M7.1–M7.2	Jobs, CronJobs	L2	migration ও scheduled task এ লাগবে	indexed jobs
M7.3	DaemonSets	L1	চিনলেই চলবে, সাধারণত লিখবেন না	—
M8.1–M8.2	RBAC, ServiceAccounts	L2	নিজের app ও CI এর permission ঠিক করবেন	aggregated ClusterRoles, authn webhook
M8.3	securityContext, PSS	L2	প্রতিটা app এ লাগবে	seccomp/AppArmor custom profile
M8.4	NetworkPolicies	L2	নিজের app এর policy লিখবেন	Cilium L7 policy
M8.5	Production secrets	L2	team এর secret workflow বুঝতে ও ব্যবহার করতে	Vault administration
M8.6	Supply chain	L1	CI তে scan বসাতে জানলেই হবে	admission-time signature enforcement
M9.1–M9.3	Kustomize, Helm	L2	প্রতিদিনের deploy tool	Helm plugin/SDK
M9.4–M9.5	CI/CD, GitOps	L2	আপনার deploy এভাবেই হবে	Argo CD multi-cluster, ApplicationSets
M9.6	Progressive delivery	L1	ধারণা থাকলেই যথেষ্ট শুরুতে	Argo Rollouts analysis

ID	Topic	Level	কেন এই depth	এখন postpone করা যায়
M10.1–M10.2	Troubleshooting	L3	সবচেয়ে মূল্যবান real-world skill	—
M10.3–M10.4	Logging, metrics	L2	dashboard/alert ব্যবহার ও নিজের app instrument করবেন	Prometheus HA, long-term storage
M10.5	Tracing	L1	পরে লাগবে	—
M11.1, M11.2, M11.4, M11.5, M11.7	Managed K8s, HA, upgrades, DR, environments	L2	app owner হিসেবে সিদ্ধান্তে অংশ নেবেন	node image management
M11.3, M11.6	Cluster autoscaling, cost	L1	platform team চালায়; প্রভাব বুঝলেই হবে	Karpenter tuning
M12.1–M12.5, M12.8	Advanced/specialist	L1	দরকার হলে পরে গভীরে	—
M12.6	Cluster from scratch, etcd ops	L4	platform engineer/CKA ছাড়া দরকার নেই	পুরোটাই postpone
M12.7	Networking internals	L4	network specialist ছাড়া দরকার নেই	পুরোটাই postpone
M13.x	Capstone	L3	সব একসাথে প্রমাণ করার জায়গা	—

নিরাপদে postpone করা যায় (প্রথম ৬ মাস): P12 এর সব, M11.3, M11.6, M10.5, M9.6, M8.6, আর যেকোনো "implementation internals"। **কখনো skip করবেন না:** M1.3, M2.3, M3.1, M3.2, M5.1, M5.2, M5.6, M10.1–M10.2।

G. Hands-on labs ও project milestones

সাতটা project, প্রতিটা আগেরটার উপর দাঁড়ানো — শেষে একই SubTrack codebase একটা production-oriented system হয়ে যাবে। প্রতিটা module এর নিজস্ব guided lab, practice task আর debugging exercise তো থাকছেই; project গুলো phase শেষের checkpoint।

Project	কখন	মূল Kubernetes resources
1. প্রথম app ও Pod inspection	P1 শেষে	Pod
2. Node.js API: Deployment + Service	P2 শেষে + M3.1	Deployment, Service
3. Configuration	P3 শেষে	ConfigMap, Secret
4. PostgreSQL ও persistent storage	P4 শেষে	StatefulSet, PVC, StorageClass, headless Service
5. Production-like networking ও reliability	P5–P7 শেষে	Probes, resources, HPA, PDB, Gateway API, Job, CronJob
6. Helm, CI/CD, monitoring, troubleshooting	P8–P10 শেষে	Helm chart, RBAC, NetworkPolicy, Argo CD, Prometheus
Final. SubTrack Capstone	P13	সব কিছু

Project 1 — প্রথম app deploy ও Pod inspection

- **Requirements:** kind এ ৩-node cluster; public `nginx` (বা `hashicorp/http-echo`) image দিয়ে একটা Pod; Pod এর IP, node, events, logs বের করা; Pod delete করে কী হয় দেখা (ফেরত আসে না — কেন?)।
- **Architecture:** ১টা Pod, কোনো Service নেই; `kubectl port-forward` দিয়ে access।
- **Likely failures:** ভুল image tag → ImagePullBackOff; port-forward এ ভুল port।
- **Acceptance:** হাতে লেখা Pod YAML (copy না); প্রতিটা field ব্যাখ্যা; `get`, `describe`, `logs`, `exec` দিয়ে Pod এর সব তথ্য বের করা; M1.3 এর ৭ ধাপ নিজের ভাষায় লেখা।

Project 2 — SubTrack API: Deployment ও Service

- **আগে লাগবে:** [DOCKER-TRACK] Dockerfile, image build, local registry বা `kind load docker-image`।

- **Requirements:** Node.js (Express/Fastify) API — `GET /plans`, `POST /subscriptions` (আপাতত in-memory), `/healthz`; ৩ replica Deployment; ClusterIP Service; একটা debug Pod থেকে DNS name দিয়ে call; v1 → v2 rolling update ও rollback।
- **Resources:** Deployment, ReplicaSet, Pod, Service, EndpointSlice।
- **Likely failures:** app `127.0.0.1` এ listen; selector mismatch → empty endpoints; ভুল `targetPort`; `latest` tag এর কারণে update না হওয়া।
- **Acceptance:** rolling update এর সময় `curl` loop এ কোনো error নেই (readiness সহ); rollback ১ মিনিটের মধ্যে; Pod delete করলে replica ফেরত আসে।

Project 3 — Configuration

- **Requirements:** `LOG_LEVEL`, `PAYMENT_MODE=mock`, `PLAN_CATALOG` (JSON file) ConfigMap থেকে; `MOCK_PROVIDER_API_KEY` (fake value) ও `DATABASE_URL` Secret থেকে; env vs file mount দুটোই; config বদলালে controlled rollout।
- **Resources:** ConfigMap, Secret, Deployment (env, envFrom, volume mount)।
- **Likely failures:** missing ConfigMap → `CreateContainerConfigError`; key নাম ভুল; config বদলেও Pod এ পুরনো value।
- **Acceptance:** কোনো secret Git এ plain text এ নেই (`.gitignore` + README তে কীভাবে তৈরি করতে হয়); dev আর prod এ একই image ভিন্ন config দিয়ে চলে।

Project 4 — PostgreSQL ও persistent storage

- **Requirements:** PostgreSQL StatefulSet (১ replica) + PVC + headless Service; SubTrack API subscriptions Postgres এ লেখে; migration একটা Job হিসেবে; Postgres Pod delete করলেও data থাকে; API তে init container বা retry logic যাতে DB ready হওয়ার আগে crash না করে।
- **Architecture:** API Deployment → `postgres` Service → `postgres-0` Pod → PVC → PV।
- **Likely failures:** PVC Pending (StorageClass); volume permission (Postgres non-root); DB password mismatch; DNS name ভুল।
- **Acceptance:** data persistence প্রমাণ (Pod delete → data আছে); pg_dump দিয়ে manual backup ও restore; একটা লেখা নোট: "production এ আমরা এটা managed DB তে রাখব কিনা ও কেন"।

Project 5 — Networking, health, resources, scaling

- **Requirements:** `/healthz` ও `/readyz` আলাদা, liveness/readiness/startup probe; request/limit; HPA (২–৬ replica); PDB; graceful shutdown (SIGTERM handler + preStop); Gateway API দিয়ে `api.subtrack.local` ও TLS (self-signed); renewal CronJob (idempotent, `concurrencyPolicy: Forbid`); worker Deployment।
- **Likely failures:** liveness এ DB check → cascading restart; HPA `<unknown>` (metrics-server/request নেই); HTTPRoute `ResolvedRefs=False`; OOMKilled।
- **Acceptance:** load test এ HPA scale out ও in; rolling update ও `kubectl drain` চলাকালীন error rate প্রায় শূন্য; CronJob দুবার চালালেও double renewal হয় না।

Project 6 — Helm, CI/CD, security, monitoring, troubleshooting

- **Requirements:** SubTrack এর Helm chart (api, worker, cronjob, migration hook, values per env); GitHub Actions: test → build → scan → push (tag = git SHA); Argo CD দিয়ে GitOps sync; namespace এ `restricted` Pod Security; default-deny NetworkPolicy + প্রয়োজনীয় allow; CI ও developer এর জন্য RBAC; Prometheus metrics, Grafana dashboard, একটা alert; Loki এ logs।
- **Likely failures:** Helm template error; Argo CD OutOfSync loop (HPA বনাম replicas); NetworkPolicy তে DNS block; restricted profile এ Pod reject।
- **Acceptance:** একটা commit থেকে ছোঁয়া ছাড়াই cluster এ নতুন version; `kubectl edit` করা drift Argo CD ফিরিয়ে দেয়; M10.2 এর ৯টা failure এর মধ্যে ৭টা hint ছাড়া সমাধান।

Final Project — SubTrack Capstone

- **Architecture:**
  - `web` — static frontend (Nginx বা Next.js)
  - `api` — Node.js REST API
  - `worker` — queue consumer (Redis queue), mock payment webhook প্রক্রিয়া করে
  - `renewal` — CronJob
  - `migrate` — pre-upgrade Job
  - `postgres` — StatefulSet (lab) / managed DB (production design এ)
  - `redis` — queue
  - Gateway API + TLS → `web` ও `api`
- **Security:** non-root, read-only rootfs, restricted PSS, default-deny NetworkPolicy, least-privilege ServiceAccount, Sealed Secrets বা External Secrets, image scan।

- **Reliability:** ≥2 replica, topology spread, PDB, graceful shutdown, probes, HPA, resource request সব Pod এ।
- **Observability:** RED dashboard, ৩টা actionable alert, centralized logs with request ID।
- **Delivery:** Helm + Argo CD, dev ও prod overlay/values, CI pipeline।
- **DR:** pg_dump CronJob → MinIO; সম্পূর্ণ নতুন cluster এ restore drill।
- **Failure drills (M13.3):** node failure, DB down, bad deploy, config error, resource exhaustion — প্রতিটার জন্য ছোট postmortem।
- **Acceptance:** K অংশের production-readiness checklist এর সব "Must" item পূরণ; একটা architecture doc; ২০ মিনিটের team presentation।

H. Estimated study time

দিনে ১ ঘণ্টা, সপ্তাহে ৬ দিন (~৬ ঘণ্টা/সপ্তাহ, revision সহ) ধরে core roadmap (P0–P11 + P13) **প্রায় ৪৪ সপ্তাহ**; ১৫% buffer যোগ করলে **১১–১২ মাস**। P12 এর specialist topic আলাদা, দরকার মতো আরও ৬–১০ সপ্তাহ।

Phase	Hours	Weeks	মোট সপ্তাহ (cumulative)	এই পর্যায়ে আপনি যা পারবেন
P0 Foundations	18–24	3–4	4	Linux/networking এর ভিত্তি
Docker track (parallel)	12–15	2–3	—	Node.js image বানাতে পারা (M2.2 এর আগে শেষ)
P1 Fundamentals	18	3	7	Architecture বোঝা, cluster চালানো
P2 Core Workloads	18	3	10	নিজের app Deployment এ চালানো
P3 Networking & Config	24	4	14	Service, DNS, config debug
P4 Storage	15–18	3	17	Postgres persistence
P5 Health & Scheduling	21–24	4	21	**Foundational competence** — zero-downtime deploy
P6 Traffic	12	2	23	Gateway API + TLS
P7 Jobs/CronJobs	9	1.5	24.5	Migration ও scheduled task

Phase	Hours	Weeks	মোট সপ্তাহ (cumulative)	এই পর্যায়ে আপনি যা পারবেন
P8 Security	18–21	3.5	28	Least-privilege setup
P9 Delivery	24–30	4.5	32.5	Helm, CI/CD, GitOps
P10 Troubleshooting & Observability	18–24	3.5	36	**Working competence** — নিজে deploy, monitor, debug
P11 Production & Cloud	24	4	40	Production আলোচনায় অংশ নিতে পারা
P13 Capstone	24–30	4.5	44.5	Portfolio-grade project
Buffer (~15%)	—	6–7	~51	অসুস্থতা, কাজের চাপ, কঠিন module এ বেশি সময়
P12 Specialist (optional)	36–60	6–10	—	বেছে নেওয়া topic

তিনটা milestone আলাদা করে দেখুন:

- **~৫ মাস (P0–P5):** foundational competence — local cluster এ নিজের app ঠিকভাবে চালাতে ও সাধারণ সমস্যা ঠিক করতে পারবেন।
- **~৮ মাস (P0–P10):** working competence — একটা real project এ team member হিসেবে Kubernetes deploy ও debug করতে পারবেন।
- **~১১–১২ মাস (Capstone সহ):** production-aware engineer। কিন্তু production experience এর বিকল্প নেই — real cluster এ ৬–১২ মাস কাজ করার পরেই আত্মবিশ্বাস সত্যিকারের হবে।

CKAD certification (optional): P10 শেষে CKAD এর syllabus প্রায় পুরোটা cover হয়ে যাবে; চাইলে ৩–৪ সপ্তাহ exam-specific practice (speed, `kubectl` shortcuts) যোগ করুন।

I. Weekly learning schedule

প্রতি সপ্তাহে ৪ দিন নতুন জিনিস, ১ দিন নিজে করা, ১ দিন debug + revision, ১ দিন ছুটি — যাতে burnout না হয় আর শেখা জিনিস মনে থাকে।

দিন	~১ ঘণ্টায় কী করবেন
দিন ১	নতুন concept: explanation পড়া + ছোট diagram নিজে আঁকা (২০ মিনিট), guided lab শুরু (৪০ মিনিট)
দিন ২	Guided lab শেষ; প্রতিটা command এর পরে "কী হলো, কেন" নোট
দিন ৩	পরের concept বা একই module এর গভীর অংশ + lab
দিন ৪	YAML হাতে লেখা (copy নিষেধ); `kubectl explain` দিয়ে field যাচাই
দিন ৫	Independent practice task — আমাকে দেখাবেন, আমি review করব
দিন ৬	Debugging exercise + review questions + আগের ২ সপ্তাহের ৫টা flashcard প্রশ্ন (spaced revision)
দিন ৭	ছুটি (বা ১৫ মিনিট হালকা: নিজের নোট পড়া)

Revision cadence:

- **প্রতি সপ্তাহ:** দিন ৬ এ আগের module গুলো থেকে ৫টা প্রশ্ন।
- **প্রতি phase শেষে:** phase checkpoint — ছোট quiz + সেই phase এর project; পাস না হলে পরের phase এ যাবেন না।
- **প্রতি ৬–৮ সপ্তাহে:** "break-and-fix" সেশন — আগের phase গুলোর মিশ্র failure।
- **Teaching:** প্রতিটা phase শেষে team কে ৫–১০ মিনিটের একটা ছোট session — শেখানোই সবচেয়ে ভালো revision।

Month-by-month plan:

মাস	Focus	শেষে deliverable
1	P0 + Docker track শুরু + M1.1–M1.2	Linux/network নোট, প্রথম Docker image
2	P1 বাকি + P2 শুরু	Project 1
3	P2 শেষ + P3 শুরু	Project 2
4	P3 শেষ + P4	Project 3, Project 4
5	P5	Foundational competence checkpoint

মাস	Focus	শেষে deliverable
6	P6 + P7 + P8 শুরু	Project 5
7	P8 শেষ + P9 শুরু	Security hardening
8	P9 শেষ + P10	Project 6, working competence checkpoint
9	P10 শেষ + P11 শুরু	Failure lab পাস
10	P11 শেষ	DR drill
11	P13 Capstone	SubTrack production-oriented deploy
12	Buffer + Capstone drills + presentation	Final assessment (K অংশ)

কোনো সপ্তাহ মিস হলে পরের সপ্তাহে দ্বিগুণ করবেন না — schedule এক সপ্তাহ পিছিয়ে দিন। Buffer এর কাজই এটা।

J. Progress checklist

একটা module তখনই tick করবেন যখন তার "Done when" নিজে প্রমাণ করেছেন, practice task আমি review করেছি, আর debugging exercise hint ছাড়া পার করেছেন। পড়ে ফেলা মানে শেষ না।

Setup (C+)

- [ ] Tools install ও যাচাই (Docker, kubectl, kind, Helm, kube-ps1)
- [ ] `kind-subtrack-lab` cluster এ ৩টা node Ready, smoke test পাস
- [ ] `lab`, `subtrack-dev`, `subtrack-prod` namespace; context safety ঠিক করা
- [ ] `~/k8s-lab` Git repo, `.gitignore` ও `reset-lab.sh` একবার চালিয়ে দেখা

Phase 0

- [ ] M0.1 Linux ও command line
- [ ] M0.2 Processes, signals, ports, permissions, env
- [ ] M0.3 Networking basics
- [ ] M0.4 Servers, VMs, containers, images
- [ ] M0.5 YAML ও shell
- [ ] M0.6 Git ও deployment concepts
- [ ] Docker track: Dockerfile, build, push, non-root image

Phase 1

- [ ] M1.1 Kubernetes কেন
- [ ] M1.2 Architecture
- [ ] M1.3 Reconciliation ও request flow
- [ ] M1.4 Local cluster
- [ ] M1.5 kubectl ও kubeconfig
- [ ] M1.6 Manifest anatomy
- [ ] **Project 1 পাস**

Phase 2

- [ ] M2.1 Pods
- [ ] M2.2 ReplicaSets ও Deployments
- [ ] M2.3 Labels, selectors, annotations
- [ ] M2.4 Namespaces
- [ ] M2.5 Rolling updates ও rollback

Phase 3

- [ ] M3.1 Services
- [ ] M3.2 DNS
- [ ] M3.3 Cluster networking model
- [ ] M3.4 ConfigMaps
- [ ] M3.5 Secrets
- [ ] **Project 2 পাস**
- [ ] **Project 3 পাস**

Phase 4

- [ ] M4.1 Volumes
- [ ] M4.2 PV, PVC, StorageClass
- [ ] M4.3 StatefulSets
- [ ] M4.4 DB on Kubernetes vs managed
- [ ] **Project 4 পাস**

Phase 5

- [ ] M5.1 Probes
- [ ] M5.2 Requests, limits, QoS
- [ ] M5.3 LimitRange ও ResourceQuota
- [ ] M5.4 Scheduling
- [ ] M5.5 Scaling ও HPA
- [ ] M5.6 PDB ও graceful shutdown
- [ ] **Foundational competence checkpoint পাস**

Phase 6–7

- [ ] M6.1 Ingress
- [ ] M6.2 Gateway API
- [ ] M6.3 TLS ও cert-manager
- [ ] M7.1 Jobs
- [ ] M7.2 CronJobs
- [ ] M7.3 DaemonSets
- [ ] **Project 5 পাস**

Phase 8

- [ ] M8.1 RBAC
- [ ] M8.2 ServiceAccounts
- [ ] M8.3 securityContext ও PSS
- [ ] M8.4 NetworkPolicies
- [ ] M8.5 Production secrets
- [ ] M8.6 Supply-chain security

Phase 9

- [ ] M9.1 Kustomize
- [ ] M9.2 Helm: ব্যবহার
- [ ] M9.3 Helm: chart লেখা
- [ ] M9.4 CI/CD
- [ ] M9.5 GitOps

- [ ] M9.6 Progressive delivery

Phase 10

- [ ] M10.1 Troubleshooting methodology
- [ ] M10.2 Failure lab (৯টার মধ্যে ≥৭)
- [ ] M10.3 Logging
- [ ] M10.4 Metrics ও alerting
- [ ] M10.5 Tracing
- [ ] **Project 6 পাস — working competence checkpoint**

Phase 11

- [ ] M11.1 Managed Kubernetes
- [ ] M11.2 HA ও reliability
- [ ] M11.3 Cluster autoscaling
- [ ] M11.4 Upgrades ও deprecated APIs
- [ ] M11.5 Backup ও DR (restore drill সহ)
- [ ] M11.6 Cost optimization
- [ ] M11.7 Multi-environment

Phase 12 (যেগুলো বেছে নেবেন)

- [ ] M12.1 Advanced autoscaling
- [ ] M12.2 Policy as code
- [ ] M12.3 Terraform/OpenTofu
- [ ] M12.4 CRDs ও Operators
- [ ] M12.5 Service mesh
- [ ] M12.6 Cluster from scratch
- [ ] M12.7 Networking internals
- [ ] M12.8 Multi-cluster ও GPU

Phase 13

- [ ] M13.1 Capstone architecture doc
- [ ] M13.2 Capstone build ও deploy

- [ ] M13.3 Failure drills ও team presentation
- [ ] **Final production-readiness assessment পাস**

K. Final production-readiness assessment

Assessment এর তিন অংশ: SubTrack এর উপর checklist audit, live failure drill, আর oral/whiteboard প্রশ্ন। তিনটাতেই পাস করলে roadmap শেষ — আর তখন আপনি real production কাজের জন্য প্রস্তুত, তবে এখনো "expert" না।

Part 1 — SubTrack checklist audit

Area	Must (সব পূরণ হতে হবে)	Should (৭০%+)
Workloads	সব app Deployment/StatefulSet/Job এ, কোনো bare Pod নেই; image tag = git SHA বা digest, `latest` নেই	recommended labels সবখানে
Health	readiness ও liveness আলাদা ও সঠিক; startup probe যেখানে দরকার	probe timeout/threshold যুক্তিসহ নথিভুক্ত
Resources	সব container এ request; memory limit; Node.js heap limit এর সাথে মেলানো	rightsizing data দিয়ে যাচাই
Availability	≥2 replica, PDB, topology spread/anti-affinity, graceful shutdown প্রমাণিত	HPA load test এ যাচাই
Config & secrets	Git এ কোনো plain secret নেই; config ও image আলাদা	Sealed Secrets/External Secrets
Security	non-root, `allowPrivilegeEscalation: false`, drop ALL capabilities, restricted PSS; default-deny NetworkPolicy; least-privilege RBAC ও ServiceAccount	read-only rootfs; image scan CI তে
Traffic	Gateway API + TLS; শুধু দরকারি Service বাইরে exposed	canary weight split ব্যবহার করা
Delivery	Helm chart + GitOps; rollback প্রক্রিয়া পরীক্ষিত; migration Job idempotent	dev/prod আলাদা values, promotion by digest
Observability	centralized logs, RED dashboard, কমপক্ষে ২টা actionable alert	request ID দিয়ে log correlation

Area	Must (সব পূরণ হতে হবে)	Should (৭০%+)
Data & DR	automated DB backup + সফল restore drill (RPO/RTO লেখা)	managed DB migration plan
Docs	architecture doc, runbook (common failures এর সমাধান)	postmortem template

Part 2 — Live failure drill (৯০ মিনিট)

আমি আপনাকে না জানিয়ে ৫টা failure inject করব (যেমন: selector typo, NetworkPolicy তে DNS block, memory limit কম, ভুল image tag, node বন্ধ)। পাস করতে হলে:

- [ ] কমপক্ষে ৪টার root cause M10.1 workflow দিয়ে বের করা
- [ ] প্রতিটা fix এক পরিবর্তনে, Git এর মাধ্যমে (production-style, `kubectl edit` না)
- [ ] প্রতিটার ৫-লাইনের postmortem: কী হলো, কীভাবে বুঝলেন, fix, ভবিষ্যতে কীভাবে আটকাবেন

Part 3 — Oral / whiteboard প্রশ্ন

1. `kubectl apply` থেকে container running পর্যন্ত প্রতিটা component এর ভূমিকা এঁকে ব্যাখ্যা করুন।
2. একটা node হঠাৎ বন্ধ হলে পরের ১০ মিনিটে cluster এ কী কী ঘটে? আপনার app এ কোনো request fail হবে কি?
3. Rolling update এ 502 আসছে — কোন ৪টা জিনিস চেক করবেন?
4. Liveness probe এ DB check দেওয়া কেন বিপজ্জনক?
5. CPU limit আর memory limit পেরোলে আলাদা কী হয়?
6. Secret কেন "encrypted" না, আর production এ কীভাবে সুরক্ষিত রাখবেন?
7. Ingress আর Gateway API এর পার্থক্য কী, আর এখন নতুন project এ কোনটা নেবেন কেন?
8. PostgreSQL কে Kubernetes এ চালাবেন নাকি managed DB — আপনার team এর জন্য যুক্তি দিন।
9. Deploy এর পর DB migration ভুল হলে rollback কেন যথেষ্ট না? expand-contract কীভাবে সাহায্য করে?
10. Upgrade এর আগে আপনার Helm chart এ কী চেক করবেন?

Scoring: Part 1 এর সব Must + Should ৭০%+, Part 2 এ ৫টার মধ্যে ৪টা, Part 3 এ ১০টার মধ্যে ৮টা সন্তোষজনক — তাহলে roadmap সম্পন্ন। যেটায় পিছিয়ে থাকবেন, সংশ্লিষ্ট module এ ফিরে যাব।

Sources

- Ingress NGINX Retirement: What You Need to Know — Kubernetes blog

- Kubernetes 1.36 release series — kubernetes.io
- Kubernetes releases & support policy — versionlog
- Migrating from ingress-nginx to the Gateway API — WZ-IT